ID du projet : .....



Université de la Manouba
École Nationale des Sciences de l'Informatique



Rapport de Stage d'Été

Sujet : Détection automatique et autocapture de la carte d'identité nationale (CIN) — Module intégré à l'application Avatar-ID

Auteur :
Mohammed Amine Chaieb



Entreprise : Codingaholic

Encadrant : Jendoubi Mohamed Khalil

Adresse : 3 Rue Mohamed Bayrem 5, La Marsa

Année universitaire : 2025/2026

Remerciements

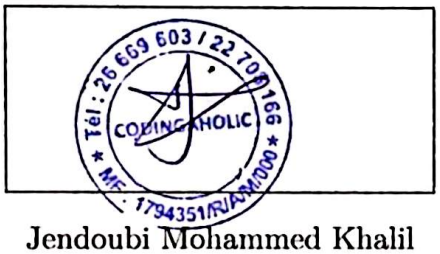
Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage.

Mes remerciements les plus sincères s'adressent en premier lieu à mon encadrant **Jendoubi Mohamed Khalil**, pour sa disponibilité, ses précieux conseils et la confiance qu'il m'a accordée tout au long de ce travail.

Enfin, je remercie mes proches pour leur soutien constant tout au long de ce parcours.

Approbation et Signatures

Encadré et approuvé par :



Jendoubi Mohammed Khalil

Résumé

Ce rapport présente le travail réalisé lors d'un stage d'été de six semaines au sein de Codingaholic, consistant à concevoir et développer un module mobile de détection automatique et d'autocapture de la carte d'identité nationale (CIN) tunisienne, intégré à l'application Avatar-ID. Le module s'appuie sur un pipeline de vision par ordinateur classique fondé sur OpenCV (détection de contours, redressement de perspective, contrôles de qualité d'image, reconnaissance recto/verso) plutôt que sur un SDK propriétaire ou un modèle entraîné sur mesure, afin de répondre aux contraintes de portabilité Android/iOS et de délai. Le développement, mené en six étapes incrémentales, a été validé par une suite de tests automatisés organisée en quatre paliers ainsi que par des tests manuels sur échantillons réels, dont les résultats détaillés sont présentés au chapitre 9.

Mots-clés : détection de document, OpenCV, autocapture, carte d'identité nationale, Flutter, vision par ordinateur, KYC.

Abstract

This report presents the work carried out during a six-week summer internship at Codingaholic, consisting of designing and developing a mobile module for automatic detection and autocapture of the Tunisian national identity card (CIN), integrated into the Avatar-ID application. The module relies on a classical computer-vision pipeline built on OpenCV (contour detection, perspective warping, image-quality checks, front/-back recognition) rather than a proprietary SDK or a custom-trained model, in order to meet the project's Android/iOS portability and time constraints. Development was carried out over six incremental stages and validated through an automated test suite organised into four tiers, complemented by manual tests on real samples, with detailed results presented in Chapter 9.

Keywords : document detection, OpenCV, autocapture, national identity card, Flutter, computer vision, KYC.

Table des matières

Liste des abréviations **9**

Introduction générale **10**

1 Présentation du projet **12**
Introduction . . . 12
1.1 Contexte du projet . . . 12
1.1.1 Présentation de l'organisme d'accueil . . . 12
1.1.2 Description du projet . . . 12
1.1.3 Problématique et objectifs . . . 13
1.2 Étude et critique de l'existant . . . 13
1.2.1 Solutions similaires sur le marché . . . 13
1.2.2 Synthèse . . . 14
1.3 Choix de la technologie de détection . . . 14
Conclusion . . . 16

2 Analyse et spécification des besoins **17**
Introduction . . . 17
2.1 Identification des acteurs . . . 17
2.2 Besoins fonctionnels . . . 17
2.3 Besoins non fonctionnels . . . 18
2.4 Diagramme des cas d'utilisation global . . . 18
2.5 Concepts théoriques et techniques . . . 19
2.6 Conception globale . . . 20
2.6.1 Architecture globale . . . 20
2.6.2 Diagrammes d'analyse . . . 21
2.7 Choix d'architecture technique et outillage . . . 23
2.8 Plan d'action . . . 23
2.9 Stratégie de test . . . 24
Conclusion . . . 24

3 Mise en place **25**
Introduction . . . 25
3.1 Objectifs de l'étape . . . 25
3.2 Conception et réalisation . . . 25
3.2.1 Architecture de la fonctionnalité `camera_capture` . . . 25
3.2.2 Cycle de vie du `CameraViewModel` . . . 26

3.2.3 Cadre-guide . . . 26
3.3 Tests . . . 26
3.4 Résultat de l'étape . . . 26

4 Détection de la carte 27
Introduction . . . 27
4.1 Objectifs de l'étape . . . 27
4.2 Conception détaillée . . . 27
4.2.1 Vue d'ensemble du pipeline . . . 27
4.2.2 Traitement en arrière-plan via Isolate . . . 28
4.2.3 Recherche du quadrilatère – `DocumentContourDetector` . . . 29
4.2.4 Redressement de perspective . . . 30
4.3 Difficultés rencontrées et corrections . . . 30
4.4 Tests . . . 30
4.4.1 Tests unitaires (module `CardQuadGeometry`) . . . 30
4.4.2 Tests dépendant d'OpenCV . . . 31
4.4.3 Tests manuels sur échantillons réels . . . 31
4.5 Résultat de l'étape . . . 31

5 Qualité d'image et autocapture 32
Introduction . . . 32
5.1 Objectifs de l'étape . . . 32
5.2 Conception détaillée . . . 32
5.2.1 Contrôle de netteté – `BlurDetector` . . . 32
5.2.2 Contrôle de luminosité – `BrightnessDetector` . . . 32
5.2.3 Contrôle de stabilité – `StabilityTracker` . . . 33
5.2.4 Logique de décision – `EvaluateCaptureReadiness` . . . 33
5.3 Tests . . . 34
5.4 Résultat de l'étape . . . 34
5.5 Paramètres et seuils retenus . . . 35

6 Reconnaissance recto/verso 37
Introduction . . . 37
6.1 Objectifs de l'étape . . . 37
6.2 Conception détaillée . . . 37
6.2.1 Détection du recto – `FaceOrientationDetector` . . . 37
6.2.2 Confirmation du recto – `TemplateMatcher` . . . 38
6.2.3 Détection du verso – `BackOrientationDetector` . . . 38
6.2.4 Enchaînement recto → verso et anti-doublon . . . 39
6.3 Tests . . . 39
6.4 Résultat de l'étape . . . 40

7 Finitions et robustesse **41**
Introduction . . . 41
7.1 Objectifs de l'étape . . . 41
7.2 Conception détaillée . . . 41
7.2.1 Recadrage définitif . . . 41
7.2.2 Écran de prévisualisation et reprise manuelle . . . 42
7.2.3 Robustesse et gestion des cas limites . . . 42
7.2.4 Limites, sécurité et confidentialité . . . 43
7.3 Tests . . . 43
7.4 Résultat de l'étape . . . 44

8 Livraison finale **45**
Introduction . . . 45
8.1 Objectifs de l'étape . . . 45
8.2 Conception détaillée . . . 45
8.2.1 Amélioration d'image – `CardImageEnhancer` . . . 45
8.2.2 Composition de la page d'impression – `PrintPageComposer` . . . 46
8.2.3 Export et stockage . . . 46
8.2.4 Historique des captures . . . 46
8.2.5 Intégration avec Avatar-ID . . . 47
8.3 Tests . . . 47
8.4 Documentation et démonstration . . . 47
8.5 Résultat de l'étape . . . 47

9 Tests et validation globale **49**
Introduction . . . 49
9.1 Organisation de la suite de tests . . . 49
9.2 Méthodes de test par composant . . . 50
9.3 Exécution de la suite de tests et lecture des résultats . . . 51
Conclusion . . . 53

Conclusion générale **54**

Bibliographie **56**

A Structure du projet **57**

B Glossaire **58**

C Protocole de capture d'écran **59**
Protection des données personnelles dans les captures . . . 60

Table des figures

2.1 Diagramme de cas d'utilisation global 19
2.2 Organisation en couches d'une fonctionnalité (exemple générique) . . . 21
2.3 Carte des modules *feature-first* de l'application 21
2.4 Diagramme de classes – entités de géométrie de détection 22
2.5 Diagramme de classes – entités de qualité et de capture 22
2.6 Séquence (1/2) – acquisition d'une frame et envoi vers l'isolate 22
2.7 Séquence (2/2) – traitement dans l'isolate et décision d'autocapture . . 23

4.1 Activité (1/2) – détection du contour et redressement 28
4.2 Activité (2/2) – contrôles de contenu et stabilisation 28

5.1 Diagramme d'état-transition – cycle de vie de l'autocapture 34

6.1 Diagramme de classes – détecteurs du recto 38
6.2 Diagramme de classes – détecteurs du verso 39
6.3 Recto avec la région d'intérêt du visage détectée 39
6.4 Verso avec les régions d'intérêt détectées 39

7.1 Étape de pivotement de l'image 42
7.2 Étape de recadrage de la carte 42
7.3 Étape d'amélioration de l'image 42

8.1 Écran de prévisualisation – recto et verso capturés 46
8.2 Écran d'historique des captures 46

9.1 Sortie terminal de `flutter test` (résumé final) 52

Liste des tableaux

1.1 Panorama des solutions existantes de vérification d'identité 14
1.2 Comparatif des trois options technologiques évaluées 16

2.1 Acteurs du système 17
2.2 Concepts théoriques et techniques mobilisés 19
2.3 Pile technologique du projet 23
2.4 Plan d'action en six étapes 24

5.1 Paramètres et seuils empiriques du pipeline 35

9.1 Paliers de la suite de tests automatisés 49
9.2 Répartition des fichiers de test automatisés 50
9.3 Méthodes de test par composant 51
9.4 Indicateurs de validation et méthode d'obtention 53

Liste des abréviations

API Application Programming Interface 9, 15

CI Intégration Continue (Continuous Integration) 9, 53

CIN Carte d'Identité Nationale 9, 11

CLAHE Contrast Limited Adaptive Histogram Equalization 9, 20

FFI Foreign Function Interface 9, 49

KYC Know Your Customer 9, 11

ROI Region of Interest 9, 39

SDK Software Development Kit 9, 15

Introduction générale

De nombreuses démarches administratives, bancaires ou commerciales exigent aujourd'hui la présentation d'une pièce d'identité, et de plus en plus souvent, sa numérisation directe depuis un smartphone : ouverture de compte en ligne, souscription à un service, vérification d'identité à distance (KYC – *Know Your Customer*), etc. Le procédé le plus courant reste la simple photo prise par l'utilisateur lui-même, une méthode rapide mais peu fiable : image floue, mal cadrée, prise à l'envers, recto capturé deux fois à la place du verso, reflet sur le plastique de la carte, etc. Ces défauts obligent ensuite soit à un contrôle humain coûteux, soit à des reprises de saisie, ce qui dégrade l'expérience utilisateur et la qualité des données collectées.

C'est dans ce contexte que s'inscrit ce stage, dont l'objectif est de concevoir et développer un module mobile de **détection automatique et d'autocapture de la Carte d'Identité Nationale (CIN) tunisienne**, intégré à **Avatar-ID**, une application de vérification d'identité à distance dans l'esprit d'une solution de KYC, mais conçue avec des exigences de sécurité renforcées afin de limiter les risques de fraude à la vérification d'identité (voir chapitre 1). Le module doit guider l'utilisateur en temps réel via le flux caméra, détecter automatiquement la présence de la carte, vérifier la qualité de l'image (netteté, luminosité, stabilité), distinguer le recto du verso, déclencher la capture sans intervention manuelle, puis recadrer et améliorer l'image obtenue – le tout en respectant des contraintes fortes de performance, de portabilité (Android **et** iOS) et de fiabilité.

Trois grandes familles de solutions techniques ont été envisagées pour la brique de détection : un service de détection de document propriétaire (Google ML Kit), un modèle de vision par ordinateur entraîné spécifiquement pour la CIN tunisienne, et une approche de vision par ordinateur classique fondée sur la bibliothèque OpenCV. Le choix final, ses justifications ainsi que les contraintes qui l'ont motivé sont détaillés au chapitre 1.

Le développement a été mené selon une démarche itérative et incrémentale, découpée en **six étapes** réparties sur environ six semaines (30 jours ouvrés), chaque étape correspondant à un livrable fonctionnel et testable. Ce découpage structure l'ensemble de ce rapport, organisé comme suit :

- Le **chapitre 1**, *Présentation du projet*, expose le contexte général, la problématique, l'étude des solutions existantes et le choix technologique retenu ;
- Le **chapitre 2**, *Analyse et spécification des besoins*, détaille les besoins fonctionnels et non fonctionnels, l'architecture logicielle globale et le plan d'action en six étapes ;
- Le **chapitre 3**, *Mise en place*, couvre la configuration du projet Flutter et l'intégration du flux caméra (étape 1) ;

- Le **chapitre 4**, *Détection de la carte*, présente le pipeline de détection de contour fondé sur OpenCV, cœur technique du projet (étape 2) ;
- Le **chapitre 5**, *Qualité d'image et autocapture*, décrit les contrôles de netteté, de luminosité, de stabilité et la logique de déclenchement automatique (étape 3) ;
- Le **chapitre 6**, *Reconnaissance recto/verso*, détaille la distinction automatique des deux faces de la carte (étape 4) ;
- Le **chapitre 7**, *Finitions et robustesse*, couvre le recadrage définitif et la gestion des cas limites (étape 5) ;
- Le **chapitre 8**, *Livraison finale*, présente le post-traitement d'image, l'export et la documentation du projet (étape 6) ;
- Le **chapitre 9**, *Tests et validation globale*, consolide la stratégie de test du projet, tous paliers confondus.

Une conclusion générale dresse enfin le bilan du travail réalisé et propose des perspectives d'amélioration.

Chapitre 1

Présentation du projet

Introduction

Ce chapitre situe le projet dans son contexte, présente la problématique traitée ainsi que les objectifs du stage, puis dresse un état de l'art des approches existantes pour la détection de document sur mobile. Il se conclut par la présentation argumentée de la solution technique retenue.

1.1 Contexte du projet

1.1.1 Présentation de l'organisme d'accueil

Le stage s'est déroulé au sein de **Codingaholic**, entreprise spécialisée dans le développement de solutions logicielles et d'applications numériques. Son siège est situé au 3 Rue Mohamed Bayrem, La Marsa, Tunisie. L'encadrement du stage a été assuré par M. Jendoubi Mohamed Khalil.

Dans le cadre de ses activités, Codingaholic développe notamment des solutions dans le domaine de l'identification numérique, parmi lesquelles figure **Avatar-ID**, une application intégrant des fonctionnalités liées à la vérification et à la gestion de documents d'identité. Le stage s'est inscrit dans ce contexte, avec pour objectif de contribuer au développement d'un module mobile de détection automatique et d'autocapture de la carte d'identité nationale tunisienne.

1.1.2 Description du projet

Le module développé au cours de ce stage s'intègre à **Avatar-ID**, une application mobile **Flutter** multiplateforme (Android et iOS) de vérification d'identité à distance. Le principe général se rapproche des solutions de **KYC** (*Know Your Customer*) utilisées par les banques et les services en ligne pour vérifier l'identité d'un utilisateur à distance, mais avec une exigence de **sécurité et de fiabilité renforcée** propre à Avatar-ID : la capture doit non seulement produire une image exploitable, mais garantir qu'il s'agit bien d'une carte physique réelle présentée en direct devant la caméra – recto et verso distincts, qualité contrôlée, sans possibilité de soumettre une simple photo d'écran ou une image déjà préparée – afin de limiter les tentatives de fraude à la vérification d'identité.

Dans ce contexte, le module objet de ce stage est chargé de capturer les deux faces

de la carte d'identité nationale tunisienne au sein d'Avatar-ID. Contrairement à une simple prise de photo manuelle, l'application doit se comporter comme un scanner intelligent : elle affiche un flux caméra en direct avec un cadre-guide, détecte la carte dès qu'elle apparaît dans le champ de la caméra, s'assure que l'image est exploitable (nette, bien exposée, stable), identifie automatiquement s'il s'agit du recto ou du verso, puis déclenche elle-même la capture – sans bouton – dès que toutes les conditions sont réunies. Une fois les deux faces capturées, l'image est automatiquement recadrée selon les contours détectés de la carte, améliorée (contraste, netteté) et présentée à l'utilisateur pour validation ou reprise.

1.1.3 Problématique et objectifs

La problématique centrale du stage peut se formuler ainsi : *comment détecter de manière fiable et en temps réel les contours d'une CIN tunisienne dans un flux caméra mobile, quelles que soient les conditions d'éclairage et d'arrière-plan, tout en respectant des contraintes de performance (traitement sous les 100 ms par image), de portabilité (Android et iOS) et de délais de développement (six semaines) ?*

Les objectifs opérationnels du stage, détaillés au chapitre 2, peuvent être résumés comme suit :

- détecter automatiquement la présence et les contours d'une CIN dans l'image caméra ;
- garantir la qualité de l'image capturée (netteté, luminosité, stabilité) ;
- distinguer automatiquement le recto du verso et enchaîner leur capture ;
- déclencher la capture sans action de l'utilisateur (autocapture) ;
- recadrer et améliorer automatiquement l'image obtenue ;
- garantir un fonctionnement identique sur Android et sur iOS.

1.2 Étude et critique de l'existant

1.2.1 Solutions similaires sur le marché

La vérification d'identité à distance est un besoin déjà couvert par plusieurs familles de solutions existantes. Le tableau 1.1 en présente les principales, avant que la section suivante n'explique pourquoi Avatar-ID a choisi de développer son propre module plutôt que d'en adopter une telle quelle.

TAB. 1.1 : Panorama des solutions existantes de vérification d'identité

Solution	**Principe**	**Limite pour Avatar-ID**
Capture manuelle simple (photo libre)	L'utilisateur cadre et déclenche lui-même la photo, sans guidage ni contrôle	Aucune garantie de cadrage, de netteté ou d'orientation ; taux de rejet élevé côté back-office, et aucune garantie que la photo soit prise en direct plutôt que réutilisée
Plateformes de KYC tierces (ex. Onfido, Jumio, Regula, IDnow)	Service cloud complet (capture guidée, vérification, parfois biométrie faciale) intégré via SDK ou API propriétaire	Dépendance à un service tiers pour un flux sensible (données d'identité transmises hors de l'application), coût de licence récurrent, personnalisation limitée pour le format spécifique de la CIN tunisienne, latence réseau pour chaque capture
Scanners de documents génériques (ex. applications de numérisation grand public)	Détection de bord de document générique, pensée pour des feuilles de papier	Non spécialisés pour une carte plastifiée de petit format ; aucune reconnaissance recto/verso ni contrôle de qualité adapté à une pièce d'identité

1.2.2 Synthèse

Les plateformes de KYC tierces couvrent une grande partie du besoin, mais imposent de faire transiter les données d'identité par un service externe et de dépendre d'une connexion réseau au moment même de la capture – deux contraintes en tension avec l'exigence de sécurité et de maîtrise des données propre à Avatar-ID (section 1.1). La capture manuelle et les scanners génériques, à l'inverse, ne garantissent ni la qualité ni l'authenticité de la capture. Aucune de ces solutions ne répond donc telle quelle au besoin : Avatar-ID a choisi de développer son propre module de détection et d'autocapture, intégré directement dans l'application et fonctionnant entièrement sur l'appareil, sans dépendance à un service tiers. Reste alors à choisir la technologie de vision par ordinateur sur laquelle fonder ce module interne, ce qui fait l'objet de la section suivante.

1.3 Choix de la technologie de détection

Trois approches techniques ont été concrètement évaluées pour réaliser ce module interne : le SDK **Google ML Kit** (module *Document Scanner*), un **modèle de vision entraîné sur mesure**, et la bibliothèque de vision par ordinateur classique **OpenCV**. Contrairement aux solutions de la section précédente, ces trois options ne sont pas

des produits concurrents mais des briques techniques parmi lesquelles choisir pour construire la fonctionnalité en interne. Le choix s'est fait par élimination, chaque option ayant été confrontée aux contraintes réelles du projet.

Option 1 – Google ML Kit (Document Scanner)

Google ML Kit propose un module de détection et de scan de documents prêt à l'emploi, avec un bon niveau de robustesse et une intégration Flutter relativement simple côté Android. Cette option a cependant été écartée pour une raison bloquante et non contournable : **le module de détection de document de ML Kit (*Document Scanner API*) n'est disponible que sur Android et ne supporte pas iOS** [15]. Le projet ayant pour exigence non fonctionnelle explicite un fonctionnement identique sur Android **et** iOS (voir chapitre 2), retenir ML Kit aurait imposé de développer une seconde chaîne de détection entièrement différente pour iOS, ce qui revient à annuler l'intérêt même d'utiliser un SDK tout-fait pour gagner du temps. Une alternative native à ML Kit du côté iOS, le framework *Vision*/*VisionKit* d'Apple, a été écartée pour la même raison de fond : elle aurait exigé de maintenir deux chaînes de détection distinctes par plateforme (Vision côté iOS, une autre solution côté Android) plutôt qu'un pipeline unique partagé, ce qu'OpenCV permet directement.

Option 2 – Modèle de vision entraîné sur mesure

Entraîner un modèle dédié (détection d'objet ou segmentation) spécifiquement sur des images de CIN tunisiennes aurait offert la meilleure robustesse potentielle, en particulier sur des arrière-plans complexes. Cette option a toutefois été écartée pour deux raisons de contraintes de stage, indépendantes de la faisabilité technique :

- **Le temps disponible n'était pas suffisant.** Un cycle complet (collecte, annotation, entraînement, validation, conversion pour mobile, intégration) dépasse largement la durée du stage (six semaines), qui devait couvrir l'ensemble de la fonctionnalité et pas uniquement la détection.
- **L'absence d'un jeu de données suffisant.** Un modèle entraîné correctement nécessite, à titre d'ordre de grandeur, plusieurs centaines d'images de CIN variées (angles, éclairages, arrière-plans) pour espérer une généralisation correcte. Or la CIN étant une pièce d'identité sensible, il n'était pas possible de collecter et d'annoter un tel volume d'images dans les délais impartis, ni de s'appuyer sur un jeu de données public équivalent pour des raisons évidentes de confidentialité.

Ces deux contraintes cumulées rendaient cette option non réalisable dans le cadre du stage, indépendamment de ses qualités techniques intrinsèques.

Option 3 – OpenCV (retenue)

Face à ces deux impasses, le choix s'est porté sur **OpenCV**, exploité via le binding Flutter `opencv_dart`, pour construire un pipeline de vision par ordinateur classique

(sans entraînement de modèle dans le cadre du stage – le classificateur en cascade de Haar utilisé au chapitre 6 est pré-entraîné, fourni par OpenCV) : détection de contours par seuillage adaptatif et algorithme de Canny, filtrage morphologique, transformation de perspective, template matching, etc. Ce choix répond directement aux deux limites précédentes :

- OpenCV est nativement multiplateforme (Android, iOS, desktop), ce qui lève le blocage identifié avec ML Kit ;
- un pipeline classique ne nécessite **aucun jeu de données annoté** ni phase d'entraînement – seul un réglage empirique de seuils sur un petit nombre d'échantillons de CIN est nécessaire, ce qui est compatible avec le délai de six semaines.

En contrepartie, cette approche demande un travail de réglage fin (seuils de Canny, scores de géométrie de quadrilatère, régions d'intérêt pour chaque élément du recto/-verso) et une robustesse qui doit être obtenue empiriquement plutôt qu'apprise – c'est précisément ce travail itératif qui est documenté dans les chapitres 3 à 8, à travers les nombreux ajustements de seuils et corrections apportés au fil des tests sur cartes réelles (fond clair/foncé, faible luminosité, reflets, etc.).

Le tableau 1.2 synthétise cette comparaison.

TAB. 1.2 : Comparatif des trois options technologiques évaluées

Critère	**ML Kit**	**Modèle sur mesure**	**OpenCV (retenu)**
Support Android	Oui	Oui	Oui
Support iOS	**Non**	Oui (si entraîné)	Oui
Jeu de données requis	Aucun	∼plusieurs centaines d'images annotées	Aucun
Compatible délai de 6 semaines	Oui	**Non**	Oui
Effort de réglage	Faible	Élevé (entraînement)	Moyen (seuils empiriques)

Conclusion

Ce chapitre a permis de situer le projet, sa problématique et ses objectifs, ainsi que de justifier le choix d'OpenCV comme fondation technique de la détection, par élimination de deux alternatives incompatibles avec les contraintes réelles du stage (portabilité iOS pour ML Kit, temps et données pour un modèle sur mesure). Le chapitre suivant détaille l'analyse des besoins et la conception globale retenue sur cette base.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre présente l'analyse des besoins fonctionnels et non fonctionnels du projet, identifie les acteurs du système, décrit l'architecture logicielle globale retenue, et détaille le plan d'action en six étapes qui structure la suite de ce rapport.

2.1 Identification des acteurs

Tab. 2.1 : Acteurs du système

Acteur	**Rôle**
Utilisateur	Personne présentant sa CIN devant la caméra pour capture (recto puis verso).
Application mobile	Système autonome pilotant la caméra, la détection, l'autocapture, le recadrage et l'amélioration d'image.
Système consommateur	Application ou service qui recevra en aval les images capturées (hors périmètre du stage).

2.2 Besoins fonctionnels

- Afficher un flux caméra en direct avec un cadre indicatif pour guider l'utilisateur.
- Détecter automatiquement la présence d'une carte CIN dans l'image.
- Vérifier que l'image est de bonne qualité (nette, bien éclairée, stable) avant de capturer.
- Reconnaître automatiquement s'il s'agit du recto ou du verso de la carte.
- Demander d'abord le recto, puis enchaîner automatiquement sur le verso.
- Déclencher la capture toute seule, dès que les conditions sont réunies, sans bouton.
- Recadrer automatiquement l'image selon les contours détectés de la carte.

- Afficher un aperçu après la capture, avec la possibilité de reprendre la photo manuellement si besoin.
- Empêcher qu'une même face soit capturée deux fois par erreur.
- Prévenir l'utilisateur si aucune carte n'est détectée après un certain temps.

2.3 Besoins non fonctionnels

- **Performance** : le traitement d'image doit rester fluide, idéalement sous les 100 ms par image.
- **Fiabilité** : le taux de bonne détection recto/verso doit atteindre au moins 90 % dans des conditions normales (éclairage ambiant intérieur suffisant, carte tenue à plat, sans reflet localisé ni angle de prise de vue extrême).
- **Compatibilité** : l'application doit fonctionner aussi bien sur Android que sur iOS (contrainte directement à l'origine du choix technologique du chapitre 1).
- **Ergonomie** : entre le moment où la carte est bien positionnée et la capture, il ne doit pas s'écouler plus de deux secondes environ.
- **Robustesse** : l'application ne doit pas planter en cas de faible luminosité, de mauvais angle ou d'absence de carte.
- **Maintenabilité** : le code doit rester organisé en modules clairs (détection, qualité, capture, interface).

2.4 Diagramme des cas d'utilisation global

Le diagramme ci-dessous synthétise les interactions entre l'utilisateur et le système autour du scénario principal (capture recto puis verso) et des scénarios secondaires (reprise manuelle, timeout, anti-doublon).

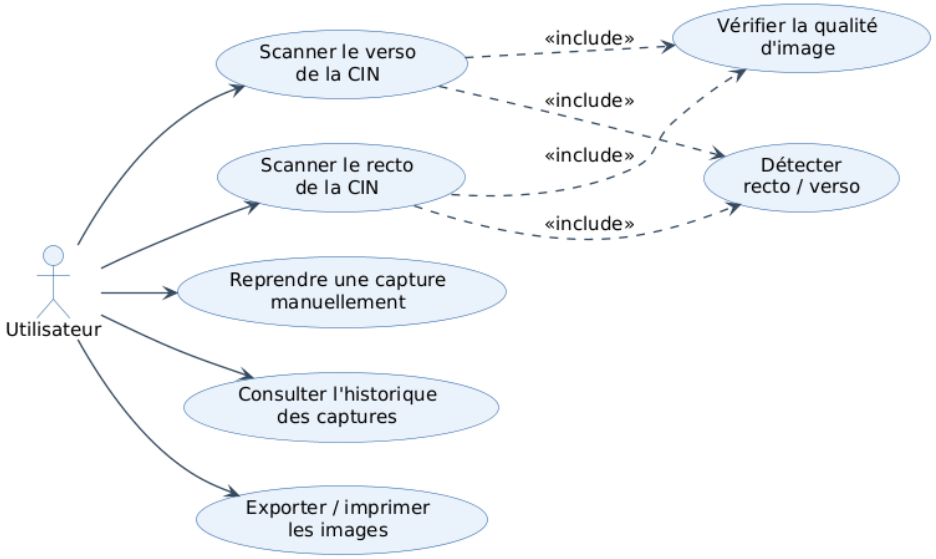


FIG. 2.1 : Diagramme de cas d'utilisation global

Note de lecture. Les cas d'utilisation « Scanner le recto » et « Scanner le verso » incluent systématiquement la vérification de qualité et la détection recto/verso, qui sont des sous-fonctions communes aux deux (relations « include »).

2.5 Concepts théoriques et techniques

Le tableau 2.2 résume les principaux concepts de vision par ordinateur mobilisés dans le projet ; ils sont détaillés au fil des chapitres correspondants.

TAB. 2.2 : Concepts théoriques et techniques mobilisés

Concept	Description
Détection de contours de Canny	Algorithme classique de détection de contours à deux seuils (bas/haut), ici dérivés dynamiquement de la médiane d'intensité de l'image plutôt que fixés arbitrairement, afin de s'adapter aux scènes claires comme sombres [1].
Seuillage adaptatif (*Adaptive Threshold*)	Binarisation d'image dont le seuil est recalculé localement (voisinage gaussien) plutôt que globalement, utile lorsqu'une carte claire se détache faiblement d'un fond clair [2].
CLAHE (*Contrast Limited Adaptive Histogram Equalization*)	Égalisation d'histogramme appliquée localement par tuiles avec limitation du contraste, pour faire ressortir des contours faibles sans amplifier excessivement le bruit [3].

Concept	Description
Transformation de perspective (*Homography / Warp*)	Application d'une matrice de transformation géométrique permettant de redresser un quadrilatère détecté (les 4 coins de la carte) en une image rectangulaire à plat, vue de face [4].
Variance du Laplacien	Mesure classique de netteté d'image : une image floue a des contours lissés donc une variance faible, une image nette une variance élevée [5].
Transformée de Hough probabiliste	Détection de segments de droites dans une carte de contours, utilisée ici pour repérer la ligne de séparation horizontale imprimée au verso de la CIN [6].
Classificateur en cascade de Haar	Détecteur d'objets classique (ici appliqué à la détection de visage) fondé sur une cascade de classificateurs faibles entraînés sur des caractéristiques de type Haar [7].
Template matching	Recherche de la meilleure corrélation entre une image de référence (logo, drapeau) et une région de l'image analysée, utilisée pour confirmer l'orientation et le contenu du recto [8].

2.6 Conception globale

2.6.1 Architecture globale

L'application est structurée selon les principes de la **Clean Architecture** [11], déclinés en architecture *feature-first* : plutôt que d'organiser le code par couche technique au niveau global (un seul dossier `screens`, un seul dossier `services`, etc.), le projet regroupe le code par fonctionnalité métier (`document_detection`, `image_quality`, `autocapture`, `camera_capture`, `image_postprocessing`, `result_preview`, `capture_history`), chaque module reprenant les couches qui lui sont utiles parmi les trois suivantes :

- **presentation** : écrans (`screens`) et view-models exposant l'état à l'interface (`ChangeNotifier`) ;
- **domain** : entités métier immuables et cas d'usage, sans dépendance à Flutter ni à OpenCV, ce qui les rend testables en isolation (voir chapitre 9) ;
- **data** : sources de données concrètes (détecteurs OpenCV, services caméra, stockage) et implémentations des dépôts (*repositories*) définis côté domaine.

Ce choix d'architecture *feature-first* a été retenu pour trois raisons directement liées au contexte du stage. D'abord, il fait correspondre naturellement l'organisation du code au plan d'action en six étapes (section 2.8) : chaque étape du stage a essentiellement consisté à livrer un ou deux modules *feature* complets, ce qui rend l'avancement du projet directement lisible dans l'arborescence du code. Ensuite, il limite les effets de bord entre fonctionnalités : modifier les seuils de détection de netteté dans `image_quality` ne

peut pas casser silencieusement la logique de recadrage dans `image_postprocessing`, puisque les deux modules ne partagent que les interfaces de domaine explicitement exposées. Enfin, il facilite la testabilité par couche : la séparation stricte entre `domain` (logique pure) et `data` / `presentation` (dépendances externes) permet de tester unitairement la plus grande partie de la logique métier sans caméra, sans OpenCV et sans émulateur, ce qui explique la proportion importante de tests de palier A dans la suite de tests (chapitre 9).

Un module transverse `core/services` regroupe les services partagés (accès caméra, chargement des ressources de classification, stockage des captures), et un module `shared/utils` regroupe les utilitaires communs (conversion d'image caméra, stabilisation des signaux de détection).

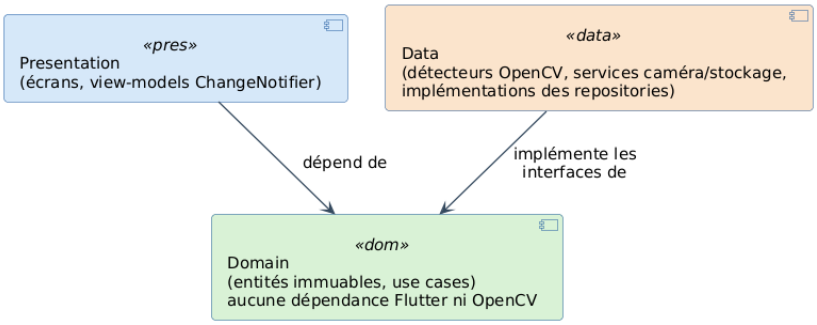


FIG. 2.2 : Organisation en couches d'une fonctionnalité (exemple générique)

Chaque fonctionnalité implémentée du projet reprend ce même motif. La figure 2.3 positionne les sept modules *feature-first* du projet.

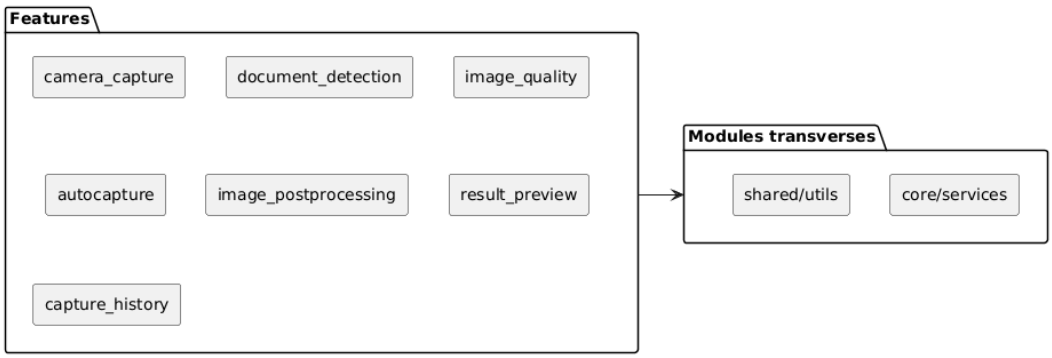


FIG. 2.3 : Carte des modules *feature-first* de l'application

2.6.2 Diagrammes d'analyse

Le diagramme de classes ci-dessous positionne les principales entités du domaine (DetectedDocument, CardQuad, QualityReport, CaptureState, CapturedCard). Pour rester lisibles, les figures ci-dessous se limitent aux entités elles-mêmes ; les cas d'usage et view-models qui assurent leur orchestration (EvaluateCaptureReadiness, DetectionViewModel, AutocaptureViewModel, CapturedCardsViewModel) ne sont volontairement pas représentés ici et sont décrits dans le texte des chapitres 4 à 6. Pour rester

lisible, ce diagramme de classes est présenté en deux parties : la figure 2.4 pour les entités liées à la géométrie de détection, et la figure 2.5 pour les entités liées à la qualité et à la capture.

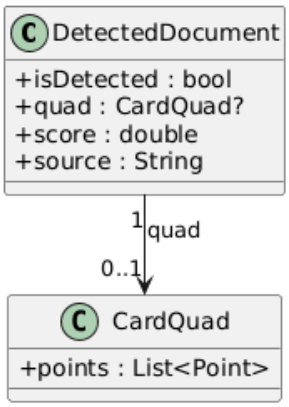


FIG. 2.4 : Diagramme de classes – entités de géométrie de détection

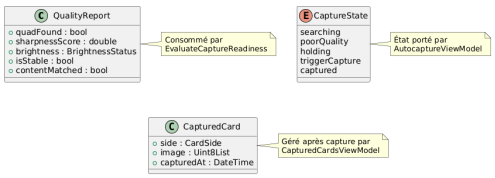


FIG. 2.5 : Diagramme de classes – entités de qualité et de capture

Le diagramme de séquence ci-après illustre le flux nominal d'une frame caméra jusqu'à la décision d'autocapture : conversion de la frame, détection du contour, calcul du rapport de qualité, puis évaluation de la disponibilité de capture.

Pour rester lisible, le flux est présenté en deux diagrammes de séquence : la figure 2.6 pour l'acquisition et l'envoi de la frame vers l'isolate, et la figure 2.7 pour le traitement dans l'isolate et la décision d'autocapture.

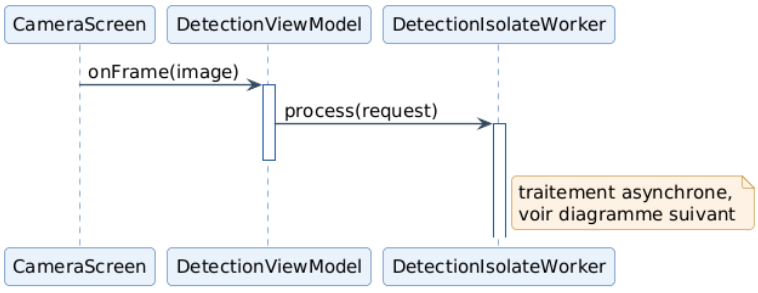


FIG. 2.6 : Séquence (1/2) – acquisition d'une frame et envoi vers l'isolate

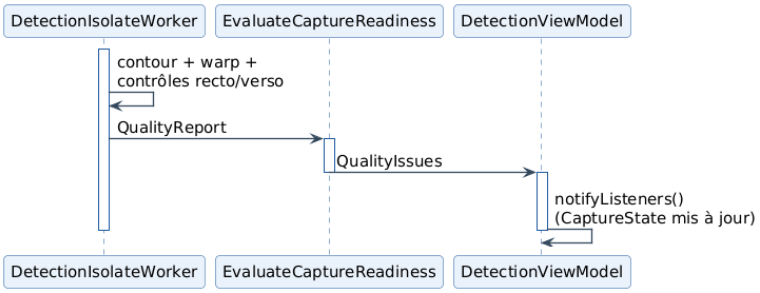


FIG. 2.7 : Séquence (2/2) – traitement dans l'isolate et décision d'autocapture

2.7 Choix d'architecture technique et outillage

Le tableau 2.3 récapitule la pile technologique retenue.

TAB. 2.3 : Pile technologique du projet

Composant	**Technologie**
Framework applicatif	Flutter (Dart) [14], cible Android et iOS
Vision par ordinateur	OpenCV [9] via le binding `opencv_dart` [13]
Traitement/encodage d'image	`package:image` (post-traitement, composition de page d'impression)
Accès caméra	`package:camera`
Permissions	`permission_handler`
Stockage / export	`path_provider`, `gal` (export vers la galerie)
Traitement concurrent	*Isolate* Dart dédié pour le pipeline OpenCV (voir chapitre 4)
Tests	`flutter_test`, `mocktail`

2.8 Plan d'action

Le développement a été réparti sur environ six semaines (30 jours ouvrés), en six étapes de durées variables selon la complexité de chacune, résumées dans le tableau 2.4 et détaillées chacune dans un chapitre dédié (chapitres 3 à 8). Les jours restants ($30 - 25{,}5 \approx 4{,}5$ jours) constituent une marge tampon utilisée pour les itérations de réglage évoquées au fil de ces chapitres.

TAB. 2.4 : Plan d'action en six étapes

Étape	Objectif	Durée	Chapitre
1	Mise en place (caméra + cadre-guide)	1,5 j	3
2	Détection de la carte en temps réel	8 j	4
3	Qualité d'image et autocapture	2 j	5
4	Reconnaissance recto/verso	4 j	6
5	Finitions et robustesse	5 j	7
6	Livraison finale (post-traitement, doc, démo)	5 j	8

2.9 Stratégie de test

Dès la phase de spécification, la stratégie de test du projet a été pensée autour de la séparation en couches retenue en section 2.6.1 : plus un composant est proche du domaine pur (logique métier, sans dépendance à Flutter ni à OpenCV), plus il est testable automatiquement et rapidement ; plus il dépend de la plateforme, de la caméra ou de contenus réels (visage, code-barres, empreinte), plus le test doit être manuel ou partiellement automatisé. Cette idée est formalisée au chapitre 9 sous la forme de quatre *paliers* de test (A à D), appliqués de façon cohérente à chaque étape du plan d'action (section 2.8) : chaque chapitre 3 à 8 documente les tests propres à son étape selon ce même découpage par palier, et le chapitre 9 en consolide la vue d'ensemble.

Conclusion

Ce chapitre a fixé le cadre fonctionnel et non fonctionnel du projet, présenté l'architecture logicielle en couches retenue et détaillé le plan d'action en six étapes qui structure les chapitres suivants. Le chapitre 3 ouvre la réalisation avec la première étape : la mise en place du projet et du flux caméra.

Chapitre 3

Mise en place

Introduction

Cette première étape (1,5 jour) vise à obtenir une application minimale affichant le flux caméra en direct avec un cadre-guide, base sur laquelle viendront se greffer la détection puis l'autocapture.

3.1 Objectifs de l'étape

- Configuration du projet Flutter et des permissions caméra (Android et iOS).
- Intégration du flux caméra en direct via `package:camera`.
- Ajout d'un cadre-guide fixe à l'écran.
- Récupération d'échantillons de CIN pour les futurs tests.

3.2 Conception et réalisation

3.2.1 Architecture de la fonctionnalité `camera_capture`

La fonctionnalité `camera_capture` a été conçue pour ne porter **aucune** logique de détection : son unique responsabilité est de gérer le cycle de vie de la caméra (permissions, démarrage, arrêt) et d'exposer le flux brut de `CameraImage` aux fonctionnalités qui en ont besoin. `CameraScreen` démarre la caméra via `CameraViewModel`, puis relaie chaque frame de `CameraController.startImageStream` directement à `DetectionViewModel.onFrame` (fonctionnalité `document_detection`, voir chapitre 4). Ce découpage strict permet de garder `camera_capture` totalement agnostique du contenu détecté : le composant `CameraOverlay` qui dessine le cadre-guide (et plus tard les silhouettes recto/verso) reçoit uniquement des booléens d'état en entrée, sans aucune logique de détection propre.

La gestion des permissions s'appuie sur `permission_handler` ; deux exceptions métier dédiées, `CameraPermissionDeniedException` et `NoCameraAvailableException`, permettent à la couche présentation d'afficher un message d'erreur spécifique plutôt qu'un état d'échec générique.

3.2.2 Cycle de vie du `CameraViewModel`

Le `CameraViewModel` pilote l'initialisation de la caméra en trois temps : demande de la permission caméra via `Permission.camera.request()` (levant une `CameraPermissionDeniedException` en cas de refus), recherche d'une caméra disponible via `availableCameras()` (levant une `NoCameraAvailableException` si aucune n'est trouvée), puis initialisation du `CameraController` en résolution haute et démarrage du flux d'images via `startImageStream`, chaque frame étant ensuite relayée telle quelle à la fonction de rappel fournie par l'appelant (`DetectionViewModel`, voir chapitre 4).

3.2.3 Cadre-guide

Le `CameraOverlay` affiche un cadre fixe au format d'une CIN (proportion standard ID-1 [12], 85,60 mm × 53,98 mm) centré à l'écran, avec un contraste suffisant pour rester visible sur fond clair comme sur fond sombre. Ce cadre sert de repère visuel à l'utilisateur avant même que la détection (chapitre 4) ne soit disponible.

3.3 Tests

Les tests de cette étape sont exclusivement manuels, la fonctionnalité ne comportant pas encore de logique métier testable en isolation :

- vérification sur téléphone réel de l'absence de lag ou de coupure du flux caméra ;
- vérification du bon comportement lorsque la permission caméra est refusée (`CameraPermissionDeniedException` affichée correctement) ;
- vérification de l'affichage correct du cadre-guide sur différentes tailles d'écran.

Les tests automatisés de cette fonctionnalité (`camera_viewmodel_test.dart`, palier B – voir chapitre 9) ont été ajoutés a posteriori, une fois la structure stabilisée, en s'appuyant sur des doubles de test (*fakes*) pour `CameraPlatform.instance` et pour `PermissionHandlerPlatform.instance`.

3.4 Résultat de l'étape

À l'issue de cette étape, l'application dispose d'une caméra fonctionnelle, testée sur un téléphone réel, avec gestion propre des permissions et un cadre-guide statique. Un premier lot d'échantillons de CIN a également été rassemblé pour servir de base aux réglages empiriques des étapes suivantes.

L'étape 2, la plus longue du projet, s'attaque au cœur du sujet : la détection automatique des contours de la carte en temps réel.

Chapitre 4

Détection de la carte

Introduction

Cette étape (8 jours, la plus longue du plan d'action) constitue le cœur technique du projet : détecter, image par image, le quadrilatère correspondant aux contours de la CIN dans le flux caméra, quelles que soient les conditions de fond et d'éclairage.

4.1 Objectifs de l'étape

- Mise en place du pipeline de détection de document fondé sur OpenCV (justifié au chapitre 1).
- Mise en place d'un retour visuel (cadre rouge/vert selon la détection).
- Premiers tests sur les échantillons récupérés à l'étape 1.

4.2 Conception détaillée

4.2.1 Vue d'ensemble du pipeline

La détection est pilotée par `DetectionViewModel`, qui reçoit chaque `CameraImage` via `onFrame` et orchestre quatre étapes :

1. conversion de la frame caméra (format YUV420/BGRA selon la plateforme) en `Mat` OpenCV, via `shared/utils/camera_image_converter.dart` ;
2. recherche du quadrilatère de la carte par `DetectionRepository` (couche domaine), implémentée par `DetectionRepositoryImpl` qui délègue à `DocumentDetectionDataSource` ;
3. redressement (*warp*) du quadrilatère détecté en une image à plat, puis exécution des contrôles propres au recto ou au verso (chapitre 6) ;
4. lissage de chaque résultat booléen brut via le `DetectionStabilizer` (`shared/utils`), afin que l'interface n'affiche « détecté » qu'après plusieurs frames consécutives confirmant la détection.

Ce pipeline est présenté en deux diagrammes d'activité successifs pour rester lisible : la figure 4.1 pour la partie détection/redressement, et la figure 4.2 pour la partie contrôle de contenu et stabilisation.

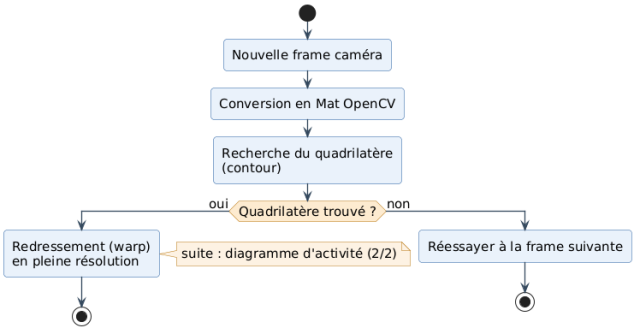


FIG. 4.1 : Activité (1/2) – détection du contour et redressement

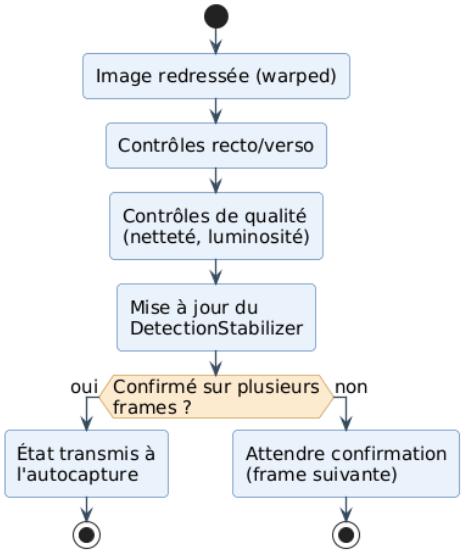


FIG. 4.2 : Activité (2/2) – contrôles de contenu et stabilisation

4.2.2 Traitement en arrière-plan via Isolate

Le traitement OpenCV étant relativement coûteux, il ne peut pas s'exécuter sur le thread principal sans bloquer le rendu de l'aperçu caméra. La première version du pipeline exécutait ce traitement de façon synchrone sur le thread de la plateforme – celui-là même qui pilote le rendu de l'aperçu caméra – ce qui causait des saccades visibles exactement proportionnelles au temps de traitement d'une frame. La solution retenue consiste à déporter l'ensemble du pipeline (conversion, détection, contrôles) sur un **isolate Dart persistant** (`DetectionIsolateWorker`), initialisé une seule fois avec le classificateur en cascade et les images de référence (logo, drapeau), puis sollicité frame par frame via un protocole de messages avec identifiant de corrélation, chaque requête étant associée à sa réponse.

Les frames ne sont pas mises en file d'attente : `DetectionViewModel` maintient un simple indicateur `_busy` et ignore toute nouvelle frame caméra tant que la précédente n'a pas reçu de réponse de l'isolate. Le débit d'analyse s'auto-régule donc à la vitesse réelle de traitement de l'isolate plutôt que d'accumuler un retard : si une frame met plus de temps que la normale à être traitée, les frames caméra suivantes sont sim-

plement abandonnées (jamais envoyées) jusqu'à ce que l'isolate redevienne disponible, sans minuterie dédiée.

4.2.3 Recherche du quadrilatère – `DocumentContourDetector`

Le détecteur de contour fonctionne selon deux modes :

- **recherche pleine image** (`_detectFullFrame`), utilisée tant qu'aucun quadrilatère n'est suivi – une passe grossière sur une image réduite (facteur 0,45) permet une première localisation rapide, puis une seconde passe en pleine résolution affine le résultat dans une marge autour de la zone trouvée ;
- **recherche suivie** (`_detectAroundTrackedQuad`), utilisée une fois un quadrilatère trouvé – la recherche se limite à une marge de 20 % autour du quadrilatère de la frame précédente, ce qui est nettement moins coûteux et évite le tremblement (*jitter*) d'une frame à l'autre.

Une fois un masque de contours obtenu (par l'une des quatre passes décrites ci-dessous), chaque contour candidat est transformé en quadrilatère par `cv.findContours` (qui extrait les contours du masque binaire) puis `cv.approxPolyDP` (qui simplifie chaque contour en un polygone, avec une tolérance – ε – de 2 % de son périmètre). Si le polygone simplifié compte exactement quatre sommets convexes, ceux-ci sont directement retenus comme quadrilatère ; sinon, un rectangle englobant minimal (`cv.minAreaRect`) est utilisé à la place, et le score du candidat est alors pénalisé (facteur 0,85) pour refléter cet ajustement moins fidèle au contour réel.

Pour générer le masque de contours dont partent ces candidats, quatre passes de traitement d'image indépendantes sont combinées : détection de contours de Canny (avec seuils dérivés de la médiane d'intensité), seuillage adaptatif gaussien, segmentation par saturation HSV (Otsu [10]), et gradient morphologique (`MORPH_GRADIENT`). Cette dernière passe est la plus coûteuse et sert de dernier recours pour les scènes où une carte et son fond sont presque identiques en niveau de gris, y compris après CLAHE : contrairement aux seuils globaux (Canny, Otsu classique), le gradient morphologique répond localement à un écart de contraste même très faible. Elle ne s'exécute que si les passes précédentes n'ont pas produit de candidat suffisamment bon (*early exit* au-delà d'un score de 0,65), ce qui limite le coût moyen par frame.

Chaque candidat retenu est ensuite noté par `CardQuadGeometry.quadScore`, une classe de géométrie pure (sans état OpenCV) qui pondère à 80 % un score de proportion et à 20 % un score de taille relative, selon :

$$\text{score}_{\text{quad}} = 0{,}8 \cdot \text{score}_{\text{proportion}} + 0{,}2 \cdot \text{score}_{\text{taille}} \tag{4.1}$$

où $\text{score}_{\text{proportion}} = \max\big(0,\ 1 - \min(|r - r_0|,\ |1/r - r_0|)\big)$, avec r le rapport largeur/hauteur mesuré du quadrilatère et $r_0 = 1{,}586$ le rapport cible du format ID-1 (85,6/54,0) – le terme en $1/r$ rend le score insensible à une inversion largeur/hauteur (quadrilatère détecté à 90°). Le score de taille compare l'aire du quadrilatère à une aire idéale visée de 30 % de la frame : il croît linéairement de 0 à 1 tant que l'aire mesurée est

inférieure à cette cible, puis redécroît linéairement au-delà. Un candidat n'est retenu comme détection que si son score dépasse le seuil `kMinDetectionScore = 0,30`. Les quadrilatères dont un coin touche le bord de l'image, ou dont l'aire dépasse 75 % de la frame (garde-fou contre une détection qui « verrouillerait » tout l'arrière-plan), sont rejetés avant même le calcul du score.

4.2.4 Redressement de perspective

Une fois le quadrilatère validé, `PerspectiveWarper` calcule la matrice d'homographie entre les quatre coins détectés et un rectangle cible aux proportions standard de la CIN, puis applique `cv.warpPerspective` pour obtenir une image à plat, prête pour les contrôles de contenu du chapitre 6.

4.3 Difficultés rencontrées et corrections

Le développement de cette étape a mis en évidence un problème récurrent et non anticipé au démarrage : **la détection fonctionnait de façon fiable sur fond sombre mais échouait très souvent sur une carte claire posée sur un fond clair** (table blanche, feuille de papier), un scénario pourtant courant en usage réel. L'origine du problème s'est révélée double : un bug de calcul (les seuils de Canny étaient dérivés par erreur de la moyenne d'intensité de l'image plutôt que de sa véritable médiane, ce qui poussait les seuils trop haut dès que la scène était claire) et un manque de contraste local sur les scènes claires, corrigé par l'ajout d'une passe de CLAHE avant la détection de contour. Ces deux correctifs, combinés à un réglage empirique des seuils de score et d'aire du quadrilatère, ont permis de retrouver un taux de détection comparable entre fond sombre et fond clair.

4.4 Tests

4.4.1 Tests unitaires (module `CardQuadGeometry`)

`CardQuadGeometry` et `DetectionStabilizer` sont tous deux des modules Dart purs, sans aucune dépendance à OpenCV ni à la caméra. `CardQuadGeometry` calcule le score et la géométrie d'un quadrilatère déjà extrait (chapitre 4) ; `DetectionStabilizer` lisse un signal booléen bruité indépendant du quadrilatère lui-même (bord détecté, visage, logo, drapeau, code-barres, empreinte – chaque signal étant suivi sous sa propre clé), à ne pas confondre avec `StabilityTracker` (chapitre 5) qui, lui, mesure spécifiquement la dérive géométrique des quatre coins du quadrilatère d'une frame à l'autre pour la logique de maintien de l'autocapture. Ces deux modules sont couverts par des tests unitaires classiques (palier A, voir chapitre 9) portant sur l'ordonnancement des points, le calcul d'aire, et la détection de contact avec le bord de l'image.

4.4.2 Tests dépendant d'OpenCV

Les tests portant sur `DocumentContourDetector`, `PerspectiveWarper` et `CardRotator` (palier C) génèrent des images synthétiques en mémoire (rectangles et dégradés dessinés via `cv.rectangle` / `cv.line`) pour vérifier que le quadrilatère retrouvé correspond bien au rectangle synthétique dessiné, à une tolérance de position près.

4.4.3 Tests manuels sur échantillons réels

En complément des tests automatisés, chaque nouvelle passe de détection a été validée manuellement sur les échantillons de CIN récupérés à l'étape 1, à différents angles et distances, afin de mesurer un taux de détection empirique avant de passer à l'étape suivante.

4.5 Résultat de l'étape

À l'issue de cette étape, le pipeline détecte de manière fiable le quadrilatère de la carte sur la majorité des scénarios testés, avec un retour visuel (cadre rouge/vert) fonctionnel, une architecture en isolate qui préserve la fluidité de l'aperçu caméra, et un jeu de tests unitaires couvrant la géométrie pure du quadrilatère.

L'étape 3 s'appuie sur ce quadrilatère détecté pour construire les contrôles de qualité d'image et la logique d'autocapture.

Chapitre 5

Qualité d'image et autocapture

Introduction

Une fois la carte détectée (chapitre 4), encore faut-il s'assurer que l'image est réellement exploitable avant de déclencher une capture automatique. Cette étape (2 jours) ajoute les contrôles de netteté, de luminosité et de stabilité, ainsi que la logique de décision d'autocapture, indépendamment de la distinction recto/verso qui n'est traitée qu'à l'étape 4.

5.1 Objectifs de l'étape

- Ajout des contrôles de netteté, de luminosité et de stabilité.
- Construction de la logique d'autocapture (attente, validation, déclenchement).

5.2 Conception détaillée

5.2.1 Contrôle de netteté – `BlurDetector`

Le contrôle de netteté repose sur la **variance du Laplacien** : une image floue présente des contours lissés donc une variance faible, une image nette des contours marqués partout donc une variance élevée. Une passe de **CLAHE** est appliquée avant le calcul du Laplacien afin de renforcer localement le contraste et d'éviter de confondre une carte simplement peu contrastée (fond clair) avec une carte réellement floue. Le score de netteté retenu (`kMinSharpnessScore`, seuil : 120) est la variance du Laplacien calculée directement sur l'image passée au détecteur – en pratique la carte déjà redressée par `PerspectiveWarper`, donc à la résolution fixe 856×540 px de ce redressement (chapitre 4). Le seuil de 120, validé empiriquement à cette résolution, n'est donc transposable tel quel qu'à cette même résolution de redressement.

5.2.2 Contrôle de luminosité – `BrightnessDetector`

Le contrôle de luminosité s'applique sur la carte déjà redressée (`warped`), pas sur l'image entière. Il combine **deux** vérifications indépendantes plutôt qu'un simple seuil global de luminosité moyenne :

- la **luminosité moyenne** (intervalle $[60, 225]$), pour détecter une scène réelle-

ment sous-exposée ou globalement surexposée ;

- la **fraction de pixels saturés** (au-delà de 12 % de pixels $\geq 250/255$), pour détecter un reflet localisé sur le plastique de la carte, indépendamment de la luminosité moyenne – une carte blanche uniformément bien exposée peut avoir une moyenne élevée sans le moindre reflet, tandis qu'un petit reflet ponctuel peut à peine déplacer la moyenne tout en rendant une zone illisible.

La méthode `BrightnessDetector.classify` applique ces deux contrôles dans l'ordre : la carte est d'abord jugée trop sombre si la moyenne descend sous le seuil bas, puis trop lumineuse si la fraction de pixels saturés ou la moyenne dépassent leurs seuils hauts respectifs, et jugée correcte sinon.

5.2.3 Contrôle de stabilité – `StabilityTracker`

Le `StabilityTracker` est un module Dart pur (sans dépendance caméra ni OpenCV) qui mesure la dérive moyenne des quatre coins du quadrilatère détecté d'une frame à l'autre, exprimée **en proportion de la diagonale du quadrilatère lui-même** plutôt qu'en pixels absolus – un seuil en pixels n'a en effet de sens qu'à une résolution de flux caméra donnée. La stabilité n'est déclarée effective qu'après `requiredStableFrames` (3) frames consécutives sous le seuil de dérive, et n'est réinitialisée qu'après `framesToReset` (2) frames consécutives au-dessus du seuil, afin qu'une unique frame de bruit ne fasse pas retomber l'indicateur à l'état « instable » après qu'il soit devenu stable.

5.2.4 Logique de décision – `EvaluateCaptureReadiness`

La décision finale d'autocapture est isolée dans un cas d'usage du domaine, volontairement sans dépendance à OpenCV ni à la caméra, afin d'être testable en isolation : à partir d'un `QualityReport` (instantané des signaux quadFound / sharpnessScore / brightness / isStable / contentMatched), elle produit un objet `QualityIssues` qui indique, pour chaque contrôle (netteté, luminosité, stabilité, correspondance de contenu), s'il est en échec ; si aucun quadrilatère n'est trouvé, tous les contrôles sont simplement considérés non applicables.

Le cycle de vie de l'autocapture est modélisé par l'énumération `CaptureState` : `searching` → `poorQuality` → `holding` → `triggerCapture` → `captured`. C'est `AutocaptureViewModel` qui porte cet état (et non `DetectionViewModel`, qui se contente de lui transmettre le `QualityReport` de chaque frame). L'état `holding` correspond au maintien pendant lequel toutes les conditions doivent rester réunies avant le déclenchement effectif ; ce maintien n'est pas une minuterie mais un compteur de frames consécutives (`requiredGoodFrames = 2`), ce qui évite un déclenchement sur un unique instant favorable isolé (faux déclenchement).

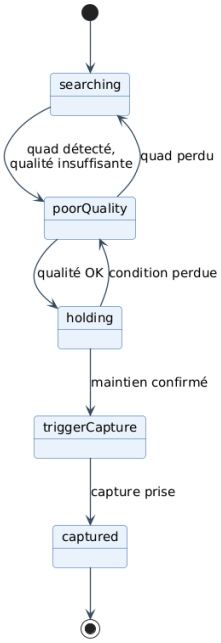


FIG. 5.1 : Diagramme d'état-transition – cycle de vie de l'autocapture

Note de lecture. Tant que le quadrilatère est détecté mais que la qualité (netteté, luminosité, stabilité) n'est pas suffisante, l'état reste à `poorQuality` ; ce n'est qu'après le délai de maintien en `holding` que la capture est effectivement déclenchée.

5.3 Tests

- **Netteté et luminosité** (palier C) : tests avec des images synthétiques volontairement floues (flou gaussien appliqué) ou sombres/surexposées, pour vérifier que `BlurDetector` / `BrightnessDetector` les rejettent bien.
- **Stabilité** (palier A) : tests unitaires de `StabilityTracker` avec des séquences de quadrilatères simulant un tremblement de main léger (dérive sous le seuil) puis un mouvement réel (dérive au-dessus du seuil), pour vérifier que la capture ne se déclenche pas trop tôt.
- **Logique de décision** (palier A) : tests unitaires de `EvaluateCaptureReadiness` et de `AutocaptureViewModel` couvrant chaque combinaison de `QualityIssues`.
- **Autocapture de bout en bout** : test manuel du positionnement jusqu'au déclenchement, en vérifiant le délai (cible : moins de deux secondes, voir besoins non fonctionnels) et l'absence de faux déclenchements.

5.4 Résultat de l'étape

À l'issue de cette étape, la capture se déclenche automatiquement, sans bouton, uniquement lorsque le quadrilatère est détecté, net, correctement exposé et stable pen-

dant plusieurs frames consécutives – sans encore distinguer le recto du verso, ce qui fait l'objet de l'étape 4.

5.5 Paramètres et seuils retenus

Le tableau 5.1 rassemble, pour référence, les principales constantes empiriques utilisées par le pipeline de détection, qualité et reconnaissance recto/verso, avec leur fichier source. Sauf mention contraire, ces valeurs ont été fixées par réglage empirique sur les échantillons de CIN disponibles (chapitre 1).

TAB. 5.1 : Paramètres et seuils empiriques du pipeline

Paramètre	**Valeur**	**Fichier source**
Score de détection minimal (`kMinDetectionScore`)	0,30	`document_contour_detector.dart`
Seuil de sortie anticipée (`_earlyExitScore`)	0,65	`document_contour_detector.dart`
Facteur de réduction, recherche pleine image (`_coarseSearchScale`)	0,45	`document_contour_detector.dart`
Marge de recherche suivie (`_trackedSearchMarginFraction`)	20 %	`document_contour_detector.dart`
Marge de recherche pleine image (`_coarseSearchMarginFraction`)	15 %	`document_contour_detector.dart`
Facteurs Canny (bas/haut) relatifs à la médiane	0,66 / 1,33	`document_contour_detector.dart`
Aire de contour minimale (`_minContourAreaFraction`)	3,5 % de la frame	`document_contour_detector.dart`
Tolérance `approxPolyDP` (`_polygonApproxEpsilonFraction`)	2 % du périmètre	`document_contour_detector.dart`
Pénalité de score, quadrilatère non net (`_nonCleanQuadScorePenalty`)	×0,85	`document_contour_detector.dart`
Biais d'*inset* du quadrilatère (`_outwardBiasInsetFraction`)	1,5 %	`document_contour_detector.dart`
Aire maximale acceptée (`_maxAreaFraction`)	75 % de la frame	`document_contour_detector.dart`
CLAHE, passe grossière / affinée (`clipLimit`)	3,5 / 2,5	`document_contour_detector.dart`
Pondération score de proportion / de taille (Éq. 4.1)	0,8 / 0,2	`card_quad_geometry.dart`
Rapport largeur/hauteur cible (`kCardAspectRatio`, ID-1)	1,586 (85,6/54,0)	`card_quad_geometry.dart`

Suite à la page suivante

TAB. 5.1 – suite de la page précédente

Paramètre	**Valeur**	**Fichier source**
Résolution du redressement (`kWarpedCardWidth`×`kWarpedCardHeight`)	856 × 540 px	`perspective_warper.dart`
Score de netteté minimal (`kMinSharpnessScore`)	120	`blur_detector.dart`
Bornes de luminosité moyenne acceptable	[60, 225]	`brightness_detector.dart`
Fraction de pixels saturés tolérée (au-delà de 250/255)	12 %	`brightness_detector.dart`
Dérive de coin maximale (`maxCornerDriftRatio`)	2,5 % de la diagonale	`stability_tracker.dart`
Frames stables requises / de réinitialisation	3 / 2	`stability_tracker.dart`
Frames de maintien requises (`requiredGoodFrames`)	2	`autocapture_viewmodel.dart`
Confirmation / réinitialisation de signal (`DetectionStabilizer`)	2 / 3 frames	`detection_stabilizer.dart`
ROI du visage (largeur / marge haute)	35 % / 15 %	`face_orientation_detector.dart`
Haar : facteur d'échelle / voisins minimums	1,05 / 8	`face_orientation_detector.dart`
Seuil de corrélation, logo/drapeau (`TemplateMatcher`)	0,6	`template_matcher.dart`
Bande de variance, empreinte plausible	[150, 4000]	`fingerprint_presence_detector.dart`
Intensité de netteté par défaut (a, Éq. 8.1)	0,6	`card_image_enhancer.dart`

L'étape 4 introduit la reconnaissance automatique du recto et du verso de la carte, condition nécessaire pour enchaîner leur capture sans intervention de l'utilisateur.

Chapitre 6

Reconnaissance recto/verso

Introduction

Cette étape (4 jours) permet à l'application de reconnaître automatiquement quelle face de la CIN est présentée devant la caméra, d'enchaîner recto puis verso sans action de l'utilisateur, et d'empêcher qu'une même face soit capturée deux fois.

6.1 Objectifs de l'étape

- Détection du recto (présence d'un visage) et du verso (code-barres, empreinte digitale et ligne de séparation, en remplacement de l'approche par zone de texte dense initialement envisagée – voir section 6.2.3).
- Enchaînement automatique recto puis verso.
- Blocage si la même face est présentée deux fois.

6.2 Conception détaillée

6.2.1 Détection du recto – `FaceOrientationDetector`

Le recto de la CIN tunisienne se distingue par la présence d'une photo d'identité, toujours positionnée dans le quart supérieur gauche de la carte lorsqu'elle est dans le bon sens. Le `FaceOrientationDetector` exploite cette régularité : il applique un **classificateur en cascade de Haar** (`cv.CascadeClassifier`) [7] sur la région supérieure gauche de la carte redressée, **testée aux quatre rotations possibles** (0°, 90°, 180°, 270°), et retient la rotation qui produit la détection de visage la plus confiante. Le classificateur utilisé est le fichier standard `haarcascade_frontalface_default.xml` distribué avec OpenCV (`opencv/data/haarcascades`, licence BSD-3-Clause), embarqué tel quel comme *asset* Flutter – aucun entraînement spécifique à la CIN tunisienne n'a été effectué sur ce classificateur. Cette approche permet de résoudre simultanément deux problèmes : confirmer qu'il s'agit bien du recto, et déterminer automatiquement son orientation correcte, sans capteur d'orientation supplémentaire. Pour chacune des quatre rotations, la méthode `detect(warpedCard)` recadre la région supérieure gauche (35 % de la largeur, marge haute de 15 %) et y exécute le classificateur (facteur d'échelle : 1,05 ; voisins minimums : 8), avant de ne conserver que la rotation ayant produit la

détection la plus confiante.

6.2.2 Confirmation du recto – `TemplateMatcher`

Une fois l'orientation du recto déterminée, `TemplateMatcher` confirme la présence du logo officiel et du drapeau tunisien par corrélation de gabarit (*template matching*) sur des régions d'intérêt fixes (fractions de la largeur/hauteur de la carte, calées sur la mise en page d'une CIN dans le bon sens), avec un seuil de corrélation de 0,6.

6.2.3 Détection du verso – `BackOrientationDetector`

Le verso ne comportant pas de visage sur lequel s'ancrer, son orientation est déterminée différemment : `BackOrientationDetector` teste également les quatre rotations, mais retient celle qui est le mieux soutenue par la combinaison de trois contrôles de contenu spécifiques au verso :

- **`BarcodeAreaDetector`** : vérifie qu'un code-barres occupe une portion plausible du haut de la carte (localisation via `cv.BarcodeDetector`, sans décodage du contenu, non nécessaire ici) ;
- **`FingerprintPresenceDetector`** : vérifie, dans la zone bas-droite de la carte, une texture d'empreinte digitale plausible via la variance du Laplacien, comprise dans une bande $[150, 4000]$ – ni trop lisse (zone vide) ni trop bruitée (reflet, tache) ;
- **`SeparationLineDetector`** : recherche la ligne horizontale de séparation imprimée en bas de carte, en combinant une carte de contours par gradient de saturation et une carte de contours de Canny, avant d'appliquer une transformée de Hough probabiliste restreinte à la portion basse de la carte – restriction nécessaire pour éviter que d'autres segments horizontaux (lignes de texte, bord de la photo, bord du code-barres) ne dominent le vote de Hough face à la ligne recherchée, en particulier sur une carte par ailleurs « propre ».

Ce diagramme de classes est divisé en deux pour rester lisible : la figure 6.1 pour les détecteurs du recto, et la figure 6.2 pour ceux du verso.

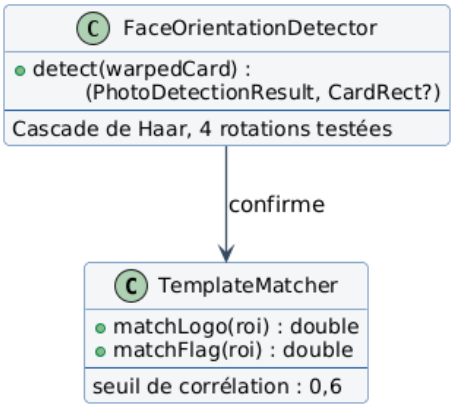


FIG. 6.1 : Diagramme de classes – détecteurs du recto

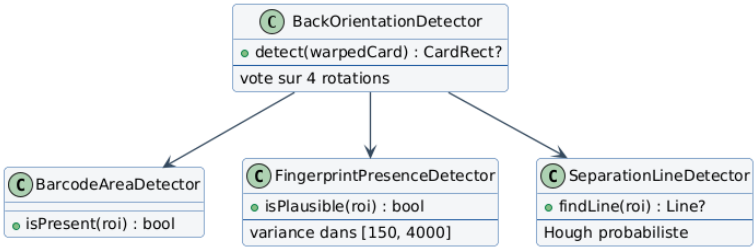


FIG. 6.2 : Diagramme de classes – détecteurs du verso

6.2.4 Enchaînement recto → verso et anti-doublon

La machine à états d'autocapture (chapitre 5) est complétée par un état de progression global (recto en attente → recto capturé → verso en attente → verso capturé) porté par `DetectionViewModel` et consommé par `AutocaptureViewModel` / `CapturedCardsViewModel`. Après la capture du recto, l'application redemande explicitement le verso ; si la même face (recto) est présentée à nouveau, la détection de contenu (présence de visage vs. code-barres/empreinte/ligne) permet de le reconnaître et de bloquer une nouvelle capture jusqu'à présentation effective du verso.

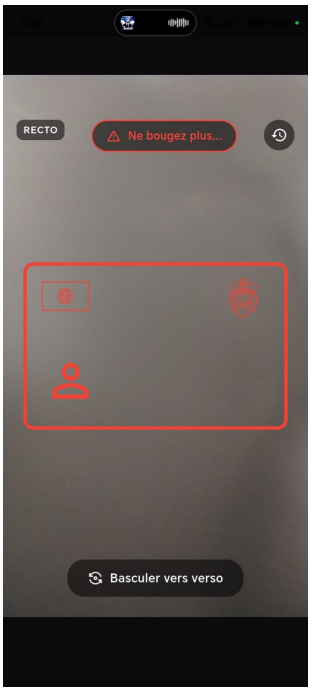


FIG. 6.3 : Recto avec la région d'intérêt du visage détectée



FIG. 6.4 : Verso avec les régions d'intérêt détectées

6.3 Tests

- **Détecteurs unitaires** (palier C) : `FingerprintPresenceDetector`, tests synthétiques sur des textures générées (bruit contrôlé, zones lisses) pour vérifier la bande de variance acceptée. `CardRotator` : vérification des quatre rotations sur images synthétiques.

- **Détection de visage et éléments réels (Haar, code-barres, empreinte, ligne)** (palier D – nécessite des images de CIN réelles et/ou un appareil) : ces contrôles dépendent de contenus réels difficiles à synthétiser de façon représentative ; ils font l'objet d'un protocole de test manuel dédié plutôt que d'assertions automatisées strictes.
- **Recto/verso de bout en bout** : test avec un mélange d'images recto et verso, pour mesurer le taux de bonne classification (cible : au moins 90 %, voir besoins non fonctionnels).
- **Anti-doublon** : test en présentant deux fois la même face, pour vérifier que l'application le détecte et redemande la bonne face.

6.4 Résultat de l'étape

À l'issue de cette étape, l'application distingue automatiquement le recto du verso, enchaîne leur capture sans intervention de l'utilisateur et empêche la capture redondante d'une même face.

L'étape 5 se concentre sur le recadrage automatique définitif de l'image et sur la robustesse générale de l'application face aux cas limites.

Chapitre 7

Finitions et robustesse

Introduction

Cette étape (5 jours) consolide l'application avant la phase de livraison : recadrage définitif de l'image capturée, comportement dans des conditions variées, et gestion propre des erreurs et cas limites.

7.1 Objectifs de l'étape

- Recadrage automatique de l'image selon les contours détectés de la carte.
- Tests dans des conditions variées (faible lumière, angles, arrière-plans différents).
- Gestion propre des erreurs et des cas limites.

7.2 Conception détaillée

7.2.1 Recadrage définitif

Le recadrage réutilise le `PerspectiveWarper` introduit au chapitre 4. Contrairement à ce que l'on pourrait supposer, le redressement n'est pas fait sur une image réduite pendant l'aperçu puis refait en pleine résolution à la capture : `DetectionViewModel` appelle `repository.warpDocument` sur la frame caméra telle quelle (`matFromFrame(frame)`), sans sous-échantillonnage préalable, aussi bien pendant l'analyse continue de l'aperçu qu'au moment de la capture – seule la recherche interne du quadrilatère (`DocumentContourDetector`, chapitre 4) travaille ponctuellement sur une image réduite (facteur 0,45) pour sa passe grossière, indépendamment du redressement lui-même. Un léger *inset* du quadrilatère (`insetQuad`, biais vers l'intérieur de 1,5 %) est déjà appliqué à chaque quadrilatère accepté par le détecteur (chapitre 4) pour éviter qu'un bord de la carte ne soit accidentellement coupé par un artefact de la détection tout en évitant à l'inverse de laisser une bordure de fond visible ; il n'est donc pas réappliqué spécifiquement à la capture.



FIG. 7.1 : Étape de pivotement de l'image



FIG. 7.2 : Étape de recadrage de la carte

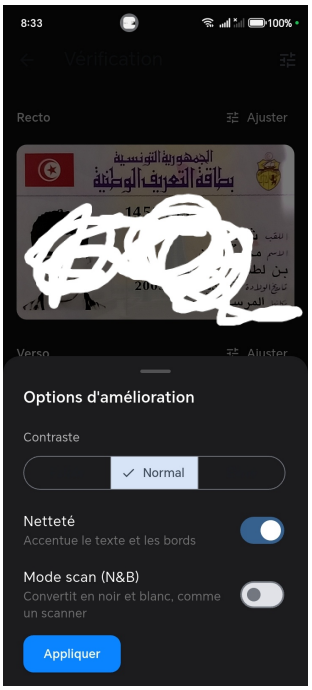


FIG. 7.3 : Étape d'amélioration de l'image

7.2.2 Écran de prévisualisation et reprise manuelle

`ResultPreviewScreen` affiche les deux faces capturées côte à côte, avec un bouton de reprise manuelle par face (`EditCaptureScreen`) qui relance le pipeline de détection pour cette face uniquement, sans perdre l'autre face déjà validée. Cette fonctionnalité répond directement au besoin fonctionnel « reprise manuelle si besoin » du chapitre 2.

7.2.3 Robustesse et gestion des cas limites

Plusieurs garde-fous ont été ajoutés pour répondre au besoin non fonctionnel de robustesse :

- **Absence de minuterie de timeout** : Le besoin fonctionnel prévoyant de prévenir l'utilisateur lorsqu'aucune carte n'est détectée après un certain délai n'est pas implémenté dans la version actuelle. Le pipeline reste fonctionnel en l'absence de carte, mais aucun message automatique n'est déclenché après une durée d'attente définie. Cette fonctionnalité constitue donc une amélioration identifiée pour une version ultérieure.
- **Libération systématique des ressources natives OpenCV** : chaque `Mat` intermédiaire créé par les détecteurs est explicitement libéré (`.dispose()`), les `Mat` étant adossés à de la mémoire native non gérée par le *garbage collector* Dart. Une attention particulière a été portée aux cas de **double libération** (par exemple lorsque `CardRotator.apply(image, 0)` ou le chemin « pas de rotation » de `PerspectiveWarper` retournent la *même* instance de `Mat` plutôt qu'une copie), désormais systématiquement détectés via `identical()` avant

tout appel à `dispose()`.

- **Isolation des erreurs de la caméra** : `CameraPermissionDeniedException` et `NoCameraAvailableException` (chapitre 3) sont capturées au niveau de l'écran et traduites en messages utilisateur explicites plutôt que de faire planter l'application.
- **Tolérance aux conditions d'éclairage** : les seuils de netteté, de luminosité et de détection de contour ont été réglés empiriquement pour rester valables aussi bien sur fond sombre que sur fond clair, et malgré des reflets localisés (voir chapitre 4, section « Difficultés rencontrées et corrections »).

7.2.4 Limites, sécurité et confidentialité

Le pipeline actuel garantit qu'une image exploitable des deux faces de la carte a été capturée en direct (recto et verso distincts, qualité contrôlée), mais il convient de préciser la portée réelle de cette garantie : les contrôles mis en place (netteté, luminosité, stabilité, reconnaissance de contenu recto/verso) réduisent fortement le risque de soumission d'une photo d'écran ou d'une image déjà préparée, sans toutefois constituer une détection d'attaque par rejeu ou de *spoofing* au sens strict (pas de détection de vivacité biométrique, pas d'analyse de texture avancée anti-fraude) ; ce volet reste hors du périmètre du stage et relève d'un travail ultérieur si le niveau de sécurité d'Avatar-ID l'exige.

Par ailleurs, deux points de confidentialité méritent d'être mentionnés explicitement :

- **Export vers la galerie** (voir section 8.2.3) : les images recto, verso et la page d'impression composée sont enregistrées dans la galerie de l'appareil via `package:gal`. Une fois exportées, ces images échappent au contrôle de l'application (elles suivent le cycle de vie normal des photos de l'appareil : sauvegarde cloud automatique, partage, etc.), ce dont l'utilisateur doit être informé.
- **Historique des captures** (voir section 8.2.4) : `capture_history` conserve les captures de la session pour consultation ;

Ces éléments doivent guider l'utilisateur d'Avatar-ID (l'équipe produit, pas l'utilisateur final) dans son évaluation du niveau de confidentialité réellement offert par le module.

7.3 Tests

- **Recadrage** : comparaison visuelle entre la carte originale et l'image recadrée, pour vérifier qu'aucun bord n'est coupé.
- **Conditions variées** : tests manuels en faible lumière, à différents angles, sur différents arrière-plans (clair, sombre, texturé).

- **Gestion des erreurs** : test en cachant la carte longtemps ou en refusant la permission caméra, pour vérifier que les messages d'erreur s'affichent correctement.
- **Double libération (`dispose`)** : les tests unitaires du détecteur de contour et du `PerspectiveWarper` (palier C) vérifient, de façon indirecte, l'absence d'exception de double-`dispose` sur les chemins « même instance » identifiés ci-dessus – il ne s'agit pas d'une mesure de fuite mémoire au sens strict (profilage d'allocation), mais d'une garantie de non-régression sur ce cas précis.

7.4 Résultat de l'étape

À l'issue de cette étape, l'application recadre correctement l'image finale, propose une reprise manuelle par face, et reste stable dans les scénarios d'erreur ou de conditions d'éclairage difficiles identifiés au cours des tests.

La sixième et dernière étape finalise l'application avec le post-traitement d'image, l'export et la documentation du projet.

Chapitre 8

Livraison finale

Introduction

Cette dernière étape (5 jours) finalise le projet : amélioration de la lisibilité des images capturées, export, petites améliorations d'ergonomie, documentation technique et préparation de la démonstration finale.

8.1 Objectifs de l'étape

- Écran de prévisualisation après capture, avec reprise manuelle possible (finalisé – base posée à l'étape 5).
- Petites améliorations d'ergonomie (animations, retours visuels).
- Rédaction de la documentation technique et du rapport de stage.
- Préparation de la démonstration finale.

8.2 Conception détaillée

8.2.1 Amélioration d'image – `CardImageEnhancer`

Une fois la carte capturée et recadrée, `CardImageEnhancer` (fonctionnalité `image_postprocessing`) applique un léger renforcement de contraste et de netteté, dans un objectif de **lisibilité** du document (texte, photo, code-barres) plutôt qu'un rendu esthétique. Le renforcement de netteté est réalisé par un noyau de convolution 3×3 de type Laplacien (et non un véritable *unsharp mask*, qui suppose un flou préalable puis une soustraction) via la fonction `img.convolution` de `package:image` – une bibliothèque Dart pure, sans dépendance native ni FFI – plutôt qu'une boucle manuelle pixel par pixel, un choix de performance nécessaire pour rester compatible avec un traitement en isolate (voir chapitre 4). Le noyau de convolution appliqué est un noyau de netteté discret classique à cinq points, qui accentue chaque pixel par rapport à ses quatre voisins directs :

$$K = \begin{pmatrix} 0 & -a & 0 \\ -a & 1+4a & -a \\ 0 & -a & 0 \end{pmatrix}, \qquad a = 0{,}6 \text{ (par défaut)} \tag{8.1}$$

plus le facteur d'intensité a est élevé, plus l'effet de netteté est marqué.

Un mode « scan » optionnel (conversion en niveaux de gris) est disponible, appliqué en dernière étape du pipeline afin que le contraste et la netteté bénéficient encore de l'information couleur complète avant l'aplatissement en un seul canal.

8.2.2 Composition de la page d'impression – `PrintPageComposer`

Pour les usages nécessitant un export imprimable, `PrintPageComposer` compose recto et verso sur une page blanche unique au format A4 portrait (2480×3508 px, soit 300 dpi), empilés verticalement avec marges – une mise en page calquée sur celle d'un scanner classique.

8.2.3 Export et stockage

`CaptureStorageService` (module `core/services`) exporte les images améliorées (recto, verso) ainsi que la page d'impression composée vers la galerie de l'appareil via `package:gal`, avec un nommage horodaté (`CIN_recto_<horodatage>`, `CIN_verso_<horodatage>`, `CIN_page_impression_<horodatage>`) garantissant l'unicité même en cas de captures rapprochées.

8.2.4 Historique des captures

La fonctionnalité `capture_history` propose un écran listant les captures précédentes de la session, permettant à l'utilisateur de les consulter ou de relancer une nouvelle capture.



FIG. 8.1 : Écran de prévisualisation – recto et verso capturés

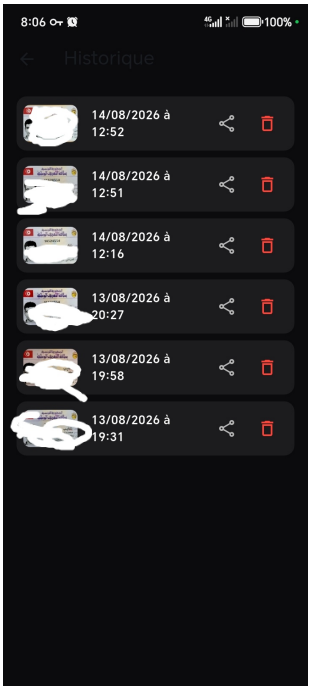


FIG. 8.2 : Écran d'historique des captures

8.2.5 Intégration avec Avatar-ID

Le module documenté dans ce rapport constitue une brique autonome au sein de l'application Avatar-ID : il prend en charge l'intégralité du parcours de capture (détection, qualité, recto/verso, recadrage, amélioration) et restitue en sortie les deux images finales (recto et verso, au format `Uint8List`) ainsi que, le cas échéant, la page d'impression composée par `PrintPageComposer`.

8.3 Tests

- **Post-traitement** (palier A, module Dart pur sans OpenCV) : `card_image_enhancer_test.dart` vérifie que le contraste/la netteté sont bien appliqués et que le mode « scan » produit une image en niveaux de gris ; `print_page_composer_test.dart` vérifie les dimensions de la page composée et le positionnement centré des deux faces.
- **View-model de post-traitement** : `postprocessing_viewmodel_test.dart` vérifie l'enchaînement amélioration → composition → état exposé à l'interface.
- **Export** (palier B, mock de `package:gal`) : `capture_storage_service_test.dart` vérifie le nommage horodaté et l'appel correct au plugin de galerie.
- **Test global de bout en bout** : scénario complet recto puis verso, sur plusieurs téléphones et dans différentes conditions de lumière. Les indicateurs agrégés disponibles sont récapitulés au chapitre 9, notamment le taux de bonne détection recto/verso et le délai de déclenchement. Le détail des résultats par appareil et par scénario n'est pas reproduit dans ce rapport.

8.4 Documentation et démonstration

La documentation technique du projet est répartie entre ce rapport et les fichiers `README.md` associés aux principales fonctionnalités (`camera_capture`, `document_detection` et `test/README.md` pour la stratégie de test détaillée au chapitre 9). Les captures d'écran présentées dans le rapport illustrent les principales étapes du parcours utilisateur ainsi que les fonctionnalités de post-traitement et d'historique. Une démonstration vidéo peut être réalisée séparément à partir du même scénario de démonstration.

8.5 Résultat de l'étape

À l'issue de cette étape, l'application est fonctionnellement complète : capture recto/verso autonome, image lisible et exportable, historique consultable, documentation technique disponible pour la prise en main du projet. Les améliorations d'ergonomie prévues se limitent à une transition animée (`AnimatedContainer`, 150 ms) sur le cadre-guide de `CameraOverlay` lors des changements d'état (absence de carte / carte détectée)

– les autres retours visuels du pipeline (messages de qualité, indicateur de maintien) avaient déjà été mis en place aux étapes précédentes (chapitre 5).

Le chapitre suivant consolide la stratégie de test globale du projet, en couvrant l'ensemble des couches de la suite automatisée ainsi que le protocole de test manuel.

Chapitre 9

Tests et validation globale

Introduction

Chaque chapitre précédent a présenté les tests propres à son étape ; ce chapitre consolide la stratégie de test du projet dans son ensemble, telle que documentée dans `test/README.md`, et rappelle les méthodes de test définies dès la phase de spécification (chapitre 2).

9.1 Organisation de la suite de tests

La suite de tests automatisés est organisée en **quatre paliers** (*tiers*), selon la manière dont chaque test peut effectivement être exécuté :

Tab. 9.1 : Paliers de la suite de tests automatisés

Palier	Nature	Environnement d'exécution
A	Logique pure Dart, sans mock ni plateforme (machines à états, entités, géométrie)	N'importe quel environnement (`flutter test test/`)
B	Dépendances de plateforme mockées (`camera`, `permission_handler`, `path_provider`, `gal`) via `mocktail`	CI standard, sans appareil ni émulateur
C	Opérations OpenCV réelles sur images synthétiques générées en test (pas de caméra, pas d'image de référence)	Nécessite le binaire natif `opencv_dart`
D	Éléments ne pouvant être vérifiés que sur données ou matériel réels (cascade de Haar, code-barres/empreinte/ligne sur vraie CIN, flux caméra live)	Appareil ou émulateur réel, avec images de référence

Le tableau 9.2 donne la répartition effective des fichiers de test du projet par palier et par fonctionnalité.

TAB. 9.2 : Répartition des fichiers de test automatisés

Fonctionnalité / module	**Fichiers**	**Exécutés**	**Cas exécutés**	**Paliers (fichiers exécutés)**
`document_detection`	7	4	59	B, C
`image_postprocessing`	7	4	24	A
`image_quality`	3	2	17	A, C
`autocapture`	3	3	26	A
`result_preview`	2	2	10	A
`camera_capture`	1	1	8	B
`core/services`	3	3	20	B
`shared/utils`	2	2	13	A, C
Total	**28**	**21**	**177**	15 (A), 6 (B), 7 (C) sur les 28 fichiers

La suite compte **28 fichiers** de test (15 de palier A, 6 de palier B, 7 de palier C). La dernière campagne d'exécution disponible (`flutter test`, reporter compact, durée totale **15 secondes**) couvre **21 de ces 28 fichiers** pour un total de **177 cas de test, tous réussis** (`00:16 +177: All tests passed!`, aucun échec). Contrairement à ce qu'une première lecture du journal pourrait laisser penser, l'absence de résultat n'est *pas* strictement liée au palier C : parmi les tests dépendant du binaire natif d'`opencv_dart` (palier C), cinq fichiers sur sept se sont bien exécutés dans cette campagne (`card_quad_geometry_test.dart`, `card_rotator_test.dart`, `fingerprint_presence_detector_test.dart`, `brightness_detector_test.dart`, `camera_image_converter_test.dart`), preuve que le binaire natif était disponible sur la machine ayant produit ce journal. Sept fichiers, répartis sur les trois paliers automatisables, n'apparaissent en revanche dans aucune ligne du journal disponible, sans message d'erreur associé : `perspective_warper_test.dart` et `blur_detector_test.dart` (palier C), `detection_repository_impl_test.dart` (palier B), ainsi que `detected_document_test.dart`, `card_image_enhancer_test.dart`, `print_page_composer_test.dart` et `enhance_captured_card_test.dart` (palier A). Aucun test de palier D n'a pu être automatisé dans le cadre du stage : ces scénarios (détection de visage réelle, code-barres/empreinte/ligne sur CIN imprimée, flux caméra de bout en bout) relèvent d'un protocole de **test manuel sur appareil**, décrit ci-après.

9.2 Méthodes de test par composant

Le tableau 9.3 reprend la méthodologie de test définie dès la phase de spécification du projet (chapitre 2), telle qu'appliquée au fil des six étapes.

TAB. 9.3 : Méthodes de test par composant

Composant	Méthode de test
Flux caméra	Test manuel sur téléphone réel, en vérifiant qu'il n'y a pas de lag ni de coupure.
Détection de la carte	Essais avec plusieurs échantillons de CIN, à différents angles et distances, pour mesurer le taux de détection ; tests unitaires de géométrie pure (palier A) et sur images synthétiques (palier C).
Netteté et luminosité	Test avec des images volontairement floues ou sombres, pour vérifier qu'elles sont bien rejetées (palier C, images synthétiques).
Stabilité	Test en simulant une dérive de position légère puis franche, pour s'assurer que la capture ne se déclenche pas trop tôt (palier A).
Recto/verso	Test avec un mélange d'images recto et verso, pour mesurer le taux de bonne classification (palier D, manuel).
Autocapture	Test de bout en bout, du positionnement jusqu'au déclenchement, en vérifiant le délai et l'absence de faux déclenchements (paliers A/B + manuel).
Recadrage	Comparaison visuelle entre la carte originale et l'image recadrée, pour vérifier qu'aucun bord n'est coupé (palier C + manuel).
Anti-doublon	Test en présentant deux fois la même face, pour vérifier que l'application le détecte et redemande la bonne face (manuel).
Gestion des erreurs	Test en cachant la carte longtemps ou en refusant la permission caméra, pour vérifier que les messages d'erreur s'affichent correctement (palier B + manuel).
Test global	Scénario complet recto puis verso, sur plusieurs téléphones et dans différentes conditions de lumière (palier D, manuel).

9.3 Exécution de la suite de tests et lecture des résultats

Chaque fonctionnalité possède son propre dossier `test/` en miroir de son dossier `lib/`, et l'ensemble de la suite s'exécute avec la commande `flutter test` (ou `flutter test test/nom_de_la_fonctionnalité/` pour ne cibler qu'une fonctionnalité). Par défaut, Flutter utilise le *reporter* `compact`, qui n'affiche qu'une ligne de progression suivie d'un résumé global du type `00:16 +177: All tests passed!` – le nombre précédé de `+` étant le compteur cumulé de cas de test individuels exécutés avec succès. C'est cette valeur, relevée sur la dernière campagne d'exécution disponible, qui sert de base au décompte de 177 cas de test du tableau 9.2 ; en cas d'échec, le compteur affiche

également le nombre d'échecs (par exemple `+175 -2`) et détaille chaque échec avec la pile d'appels correspondante – un cas qui ne s'est pas présenté dans cette campagne.

Pour obtenir un résultat plus détaillé que ce résumé global, deux options complémentaires sont disponibles :

- `flutter test --reporter expanded` affiche une ligne de résultat (`PASS` ou `FAIL`) pour *chaque* cas de test individuellement, avec son nom complet – utile pour identifier précisément quel cas échoue plutôt que de ne voir que le compteur global ;
- `flutter test --coverage` génère un rapport de couverture au format `lcov` (`coverage/lcov.info`), à partir duquel un pourcentage de lignes couvertes par fonctionnalité peut être calculé (par exemple avec `lcov --summary coverage/lcov.info` ou en générant un rapport HTML navigable avec `genhtml`).



FIG. 9.1 : Sortie terminal de `flutter test` (résumé final)

Le tableau 9.4 récapitule les indicateurs à suivre pour valider les besoins non fonctionnels du chapitre 2. Les résultats des paliers A et B, entièrement automatisables, peuvent être obtenus directement par une exécution de `flutter test` ; les résultats liés au temps de traitement par frame et aux taux de détection/classification nécessitent en complément une mesure sur appareil réel, dans le cadre du protocole de test manuel décrit à la section précédente.

TAB. 9.4 : Indicateurs de validation et méthode d'obtention

Indicateur	**Méthode d'obtention**	**Résultat obtenu**
Tests automatisés passés (21 des 28 fichiers de la suite)	`flutter test`, lecture du compteur `+N` final	**177/177 cas exécutés réussis** (0 échec, dernière campagne disponible – voir section 9.3)
Temps de traitement par frame	Mesure manuelle sur appareil (horodatage avant/après le traitement d'une frame dans `DetectionIsolateWorker`)	entre 92ms et 100ms
Taux de bonne détection recto/verso	Protocole de test manuel sur échantillons de CIN réels (palier D)	92 %
Délai positionnement → capture	Mesure manuelle chronométrée, de la stabilisation du quadrilatère au déclenchement	≈ 2 s

Conclusion

La stratégie de test retenue combine des tests automatisés rapides et déterministes (paliers A et B, exécutables sans matériel) avec des tests dépendants d'OpenCV (palier C) et un protocole de test manuel structuré pour les éléments nécessitant du matériel et des cartes réelles (palier D). Les mesures disponibles sur appareil réel sont récapitulées dans le tableau 9.4 ; certains scénarios manuels restent toutefois à étendre et à consolider sur un échantillon plus représentatif.

Conclusion générale

Ce stage a permis de concevoir et de développer, en six semaines, un module mobile complet de détection automatique et d'autocapture de la carte d'identité nationale tunisienne, intégré à **Avatar-ID**, une application de vérification d'identité à distance. Face aux contraintes réelles du projet – absence de jeu de données annoté, délai court, exigence de compatibilité Android **et** iOS – le choix d'un pipeline de vision par ordinateur classique fondé sur OpenCV, plutôt que sur un SDK propriétaire limité à Android (Google ML Kit) ou sur un modèle entraîné sur mesure impossible à constituer dans les délais, s'est révélé pertinent : il a permis un cycle de développement itératif rapide, chaque correctif de seuil ou de logique pouvant être testé en quelques heures sur les échantillons disponibles.

Le découpage en six étapes (mise en place, détection, qualité/autocapture, recto/verso, finitions, livraison) a structuré l'ensemble du travail et a permis de livrer, à chaque étape, une fonctionnalité testable de façon incrémentale. L'architecture en couches (Clean Architecture, feature-first) adoptée dès le départ a par ailleurs permis d'isoler la logique métier pure (géométrie, machine à états, décision d'autocapture) de ses dépendances techniques (OpenCV, caméra, plateforme), facilitant à la fois les tests automatisés (chapitre 9) et les nombreux ajustements empiriques nécessaires au fil des six étapes (chapitres 3 à 8).

Limites et perspectives. Plusieurs axes d'amélioration se dégagent pour la suite du projet :

- **Exécution complète de la suite de tests** : la dernière campagne disponible a couvert 21 des 28 fichiers de test, pour un total de 177 cas exécutés et réussis. Une campagne complémentaire reste nécessaire pour exécuter les sept fichiers non couverts et confirmer leur bon fonctionnement dans l'environnement de test disponible.
- **Constitution progressive d'un jeu d'images de CIN** : dans le respect des règles de confidentialité applicables, la constitution d'un jeu d'images plus représentatif permettrait d'évaluer plus largement les taux de détection et de classification et, à terme, d'envisager l'introduction d'un modèle appris en complément du pipeline OpenCV actuel.
- **Automatisation des scénarios de palier D** : les scénarios dépendant de contenus réels, notamment la détection du visage, du code-barres, de l'empreinte et de la ligne de séparation, sont actuellement couverts par un protocole de test manuel plutôt que par des assertions automatisées strictes. La constitution de fixtures d'images de référence permettrait d'automatiser une partie de ces validations.

- **Consolidation des mesures de performance et de fiabilité** : la campagne disponible fournit un temps de traitement par frame compris entre 92 ms et 100 ms, un taux de bonne détection recto/verso de 92 % et un délai positionnement → capture d'environ 2 s (tableau 9.4). Une campagne complémentaire sur davantage d'appareils, de conditions et d'échantillons permettrait de consolider la représentativité de ces mesures.
- **Ajout d'une minuterie de timeout** : le besoin fonctionnel prévoyant de prévenir l'utilisateur lorsqu'aucune carte n'est détectée après un certain délai n'est pas implémenté dans la version actuelle. Cette fonctionnalité constitue une amélioration identifiée pour une version ultérieure.

Sur le plan personnel, ce stage a été l'occasion d'approfondir des compétences en vision par ordinateur appliquée (OpenCV), en développement mobile multiplateforme (Flutter/Dart), en architecture logicielle (Clean Architecture) et en démarche de test structurée – tout en expérimentant concrètement l'importance d'un choix technologique argumenté au regard des contraintes réelles d'un projet, plutôt que du seul critère de performance théorique d'une solution.

Bibliographie

[1] J. Canny, *A Computational Approach to Edge Detection*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 1986.

[2] R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4e édition, Pearson, 2018.

[3] K. Zuiderveld, *Contrast Limited Adaptive Histogram Equalization*, Graphics Gems IV, Academic Press, 1994.

[4] R. Hartley, A. Zisserman, *Multiple View Geometry in Computer Vision*, 2e édition, Cambridge University Press, 2004.

[5] J. L. Pech-Pacheco et al., *Diatom Autofocusing in Brightfield Microscopy : A Comparative Study*, Proceedings of the International Conference on Pattern Recognition, 2000.

[6] J. Matas, C. Galambos, J. Kittler, *Robust Detection of Lines Using the Progressive Probabilistic Hough Transform*, Computer Vision and Image Understanding, 2000.

[7] P. Viola, M. Jones, *Rapid Object Detection Using a Boosted Cascade of Simple Features*, IEEE Conference on Computer Vision and Pattern Recognition, 2001.

[8] R. Brunelli, *Template Matching Techniques in Computer Vision : Theory and Practice*, Wiley, 2009.

[9] G. Bradski, A. Kaehler, *Learning OpenCV : Computer Vision with the OpenCV Library*, O'Reilly Media, 2008.

[10] N. Otsu, *A Threshold Selection Method from Gray-Level Histograms*, IEEE Transactions on Systems, Man, and Cybernetics, 1979.

[11] R. C. Martin, *Clean Architecture : A Craftsman's Guide to Software Structure and Design*, Prentice Hall, 2017.

[12] ISO/IEC 7810 :2019, *Identification cards – Physical characteristics*, International Organization for Standardization, 2019.

[13] Bibliothèque `opencv_dart`, binding Dart/Flutter pour OpenCV. https://pub.dev/packages/opencv_dart

[14] Documentation officielle Flutter. https://docs.flutter.dev

[15] Documentation Google ML Kit – Document Scanner API (disponibilité par plateforme). https://developers.google.com/ml-kit

Annexe A

Structure du projet

L'arborescence ci-dessous résume l'organisation des fonctionnalités du code source, en architecture *feature-first* (chapitre 2).

- `core/` – services transverses (`camera_service`, `capture_storage_service`, `cascade_asset_loader`) et thème de l'application ;
- `shared/` – composants d'interface communs et utilitaires partagés (`camera_image_converter`, `detection_stabilizer`) ;
- `features/` – une fonctionnalité par module, chacune organisée en couches *presentation* / *domain* / *data* (chapitre 2) :
  - `camera_capture` (presentation uniquement) ;
  - `document_detection` (détecteurs et dépôts en *data*, entités et cas d'usage en *domain*, écrans en *presentation*) ;
  - `image_quality` (détecteurs en *data*) ;
  - `autocapture` (*domain* et *presentation*) ;
  - `image_postprocessing` (les trois couches) ;
  - `result_preview` (*domain* et *presentation*) ;
  - `capture_history` (presentation uniquement).
- `test/` – suite de tests automatisés en miroir de `lib/` (28 fichiers, dont 21 couverts par la dernière campagne d'exécution disponible pour 177 cas de test tous réussis, voir chapitre 9).

Annexe B

Glossaire

Terme	Définition
CIN	Carte d'Identité Nationale (tunisienne dans le cadre de ce projet).
Quad / Quadrilatère	Les quatre coins détectés délimitant la carte dans l'image caméra.
Warp / Redressement	Transformation de perspective ramenant un quadrilatère détecté à une image rectangulaire à plat.
CLAHE	*Contrast Limited Adaptive Histogram Equalization*, égalisation de contraste locale limitée.
Isolate	Unité d'exécution concurrente de Dart (mémoire isolée), utilisée ici pour déporter le traitement OpenCV hors du thread d'interface.
Mat	Structure de données image d'OpenCV (*matrix*), adossée à de la mémoire native.
Tier (palier) de test	Classification des tests selon leur environnement d'exécution requis (A : pur Dart, B : mocks, C : OpenCV natif, D : appareil/-données réelles).
KYC	*Know Your Customer*, processus de vérification d'identité à distance.

Annexe C

Protocole de capture d'écran

Cette annexe décrit le protocole recommandé pour réaliser des captures d'écran représentatives de l'application, notamment pour les écrans impliquant le flux caméra, l'autocapture et la prévisualisation. Elle précise également les précautions à prendre afin de protéger les données personnelles éventuellement visibles sur les cartes utilisées lors des tests.

- **Android (appareil physique)** : combinaison de touches `Volume bas + Power` maintenue environ une seconde ; l'image est enregistrée dans la galerie (dossier `Screenshots`). Alternative sans intervention manuelle sur l'écran : depuis un poste connecté en USB avec le SDK Android installé, la commande `adb exec-out screencap -p > capture.png` capture l'écran sans afficher la moindre interface système, ce qui est préférable lorsque l'aperçu caméra est actif.
- **Android (émulateur, Android Studio)** : le panneau *Device Manager* affiche une icône dédiée de capture d'écran dans la barre d'outils de l'émulateur ; l'image est enregistrée directement sur le poste de développement.
- **iOS (appareil physique)** : combinaison `Power + Volume haut` (pression brève et simultanée) sur les modèles sans bouton principal.
- **iOS (simulateur, Xcode)** : menu *Device* > *Trigger Screenshot*, ou raccourci `Cmd+S` ; l'image est enregistrée sur le Bureau par défaut.
- **Flutter DevTools** : l'onglet *Flutter Inspector*, accessible pendant une session de débogage (`flutter run`), permet également d'exporter une capture de l'arbre de widgets courant, utile pour documenter un écran précis sans dépendre de l'OS.

Pour les écrans impliquant le flux caméra (détection en cours, autocapture), il est recommandé de présenter une carte devant l'objectif au moment de la capture plutôt que de simuler l'aperçu, afin que les captures d'écran reflètent fidèlement le comportement réel de l'application décrit aux chapitres 3 à 8.

Protection des données personnelles dans les captures

Les échantillons de CIN utilisés pendant le développement (chapitre 3) comportent des données à caractère personnel réelles (photo, nom, numéro de carte, code-barres). Cela ne pose pas de problème pour un usage de développement interne, mais toute capture d'écran destinée à figurer dans ce rapport ou dans tout autre document partagé (soutenance, dépôt de code, archivage) doit être traitée avant intégration, selon l'une des approches suivantes :

- **Occultation a posteriori** : appliquer un cache opaque ou un flou fort sur la photo, le nom, le numéro de carte et le code-barres avant d'exporter la capture d'écran, tout en conservant visibles les éléments propres à l'interface (cadre-guide, indicateur de détection, messages de qualité) qui sont l'objet réel de l'illustration ;
- **Carte factice dédiée** : produire, pour les seules captures destinées au rapport, une carte de démonstration au format ID-1 avec des données fictives (nom, numéro, photo générique, code-barres arbitraire), sans changer le protocole de développement ou de test interne qui peut continuer à s'appuyer sur des échantillons réels ;
- **Cadrage restreint** : limiter la capture d'écran à la seule zone d'interface utile lorsque cela suffit à illustrer le propos, en excluant la région où les champs lisibles de la carte apparaîtraient.

Cette précaution s'applique à toute capture destinée au rapport, y compris celles évoquées au chapitre 8 (écran de prévisualisation) et aux figures des chapitres 4 à 6 (détection, qualité, recto/verso).